MEG-003 — Domain-Driven Design

External publication draft

MEG-003 — Domain-Driven Design

> *Software should model the business. The business should never model the software.*

Purpose

As Mosaic grows, it will encompass many independent business capabilities.

Examples include:

- Libraries
- Metadata
- Playback
- Users
- Authentication
- Extensions
- Collections
- Recommendations
- Books
- Music
- Live TV

Attempting to model all of these concerns within a single unified object model would inevitably produce a tightly coupled, increasingly complex platform.

Domain-Driven Design (DDD) provides the architectural principles required to manage that complexity.

Unlike many implementations of DDD, Mosaic does **not** adopt Domain-Driven Design because it is fashionable.

It adopts DDD because it aligns naturally with:

- Event-Driven Runtime
- Extension-first architecture
- Hexagonal Architecture
- Autonomous capabilities
- Long-term platform evolution

Relationship to MEG

```
MEG-001

↓

Engineering Standards

↓

MEG-002

↓

Reactive Runtime

↓

MEG-003

↓

Business Modelling

↓

MEG-004

↓

Hexagonal Architecture

↓

Implementation
```

MEG-001 explains **how software is written.**

MEG-002 explains **how software executes.**

MEG-003 explains **how the business itself is modelled.**

Scope

This specification defines:

- Domain philosophy
- Ubiquitous language
- Subdomains
- Core domains
- Supporting domains
- Generic domains
- Bounded contexts
- Context maps
- Entities
- Value objects

- Aggregates
- Aggregate roots
- Domain services
- Domain events
- Repositories
- Factories
- Domain invariants

This specification intentionally does **not** define:

- Runtime behaviour
- Event delivery
- Worker execution
- Scheduling
- Transport protocols
- Storage implementation

Those concerns are defined by other MEG specifications.

Core Question

MEG-003 exists to answer one question.

> ***How should the Mosaic business domain be modelled so that complexity remains understandable as the platform grows?***

Domain Statement

Within Mosaic:

> ***Software should reflect the language of the business, not the language of the implementation.***

The domain model should describe:

- media
- libraries
- playback
- users
- metadata

It should **not** describe:

- controllers
- databases

- HTTP
- workers
- SQL

Business concepts should remain independent of technical concerns.

This is one of the central goals of Domain-Driven Design: creating a shared domain model expressed through a ubiquitous language understood by both domain experts and engineers. [oai_citation:0‡Google Books](https://books.google.com/books/about/Domain_Driven_Design_Reference.html?id=ccRsBgAAQBAJ&utm_source=chatgpt.com)

Domain Hierarchy

The Mosaic platform intentionally separates domain modelling into conceptual layers.

```
Business Domain

↓

Subdomains

↓

Bounded Contexts

↓

Aggregates

↓

Entities

↓

Value Objects

↓

Implementation
```

Each layer owns exactly one responsibility.

Future chapters define every layer in detail.

Expected Outcome

After reading MEG-003 contributors should understand:

- why Mosaic uses Domain-Driven Design
- how bounded contexts are identified
- how aggregates enforce business invariants
- how entities differ from value objects
- where domain events originate
- how repositories interact with aggregates

- how capabilities align with business domains

without discussing runtime implementation or transport infrastructure.

Repository Structure

```
engineering/
■■■ meg/
    ■■■ MEG-003 Domain-Driven Design/
        README.md
        00-document-control.md
        01-domain-philosophy.md
        02-ubiquitous-language.md
        03-subdomains.md
        04-bounded-contexts.md
        05-context-maps.md
        06-entities.md
        07-value-objects.md
        08-aggregates.md
        09-aggregate-roots.md
        10-domain-services.md
        11-domain-events.md
        12-repositories.md
        13-factories.md
        14-domain-invariants.md
        15-modelling-guidelines.md
        16-adrs.md
        17-contributor-guidance.md
        glossary.md
        references.md
```

Dependencies

Required reading:

- MEG-001 Go Engineering Standards
- MEG-002 Event-Driven Runtime
- MDL-002 Principles

- MDL-003 Mental Model

Future companion specifications:

- MEG-004 Hexagonal Architecture
- MEG-005 Extension Platform
- MEG-006 Runtime Architecture

Design Goals

The Domain Model is intended to produce software that is:

- Business focused
- Ubiquitous
- Cohesive
- Evolvable
- Loosely coupled
- Testable
- Expressive
- Independent of infrastructure

The model should become deeper as understanding improves.

It should never become more technical.

Review Status

Status

Draft

Owner

Lead Software Architect

Next File

`00-document-control.md`

Document Control

Document Information

Field	Value
Document ID	MEG-003
Title	Domain-Driven Design
File	00-document-control.md
Status	Draft
Version	0.1
Owner	Lead Software Architect
Classification	Internal Architecture Specification

Purpose

This document establishes the governance, authority and lifecycle of the Mosaic Domain-Driven Design specification.

MEG-003 defines the canonical approach to modelling business domains throughout the Mosaic platform.

Unlike implementation documentation, this specification defines how business complexity is understood, organised and communicated.

It intentionally separates business thinking from implementation thinking.

Authority

MEG-003 is the authoritative specification governing business modelling within the Mosaic ecosystem.

This specification applies to:

- Mosaic Core
- First-party Extensions
- Third-party Extensions
- SDK Development
- Runtime Capabilities
- Future Platform Features

Every business capability introduced into Mosaic SHOULD align with the modelling principles defined within this specification.

Relationship to Other Specifications

MEG specifications intentionally build upon one another.

```
MDL
↓
MDS
↓
MEG-001
↓
MEG-002
↓
MEG-003
```

Specifically:

- **MDL** defines product philosophy.
- **MDS** defines presentation.
- **MEG-001** defines engineering practices.
- **MEG-002** defines runtime behaviour.
- **MEG-003** defines business modelling.

Future specifications use the domain model established here as the foundation for architectural boundaries.

Normative Language

Unless explicitly stated otherwise, the following keywords are interpreted according to RFC 2119.

Keyword	Meaning
MUST	Mandatory requirement.
MUST NOT	Prohibited behaviour.
SHOULD	Strong recommendation. Deviation requires architectural justification.
SHOULD NOT	Discouraged except where clearly justified.
MAY	Optional behaviour based upon engineering judgement.

Examples and diagrams are informative unless explicitly identified as normative.

Domain Principles

The Mosaic domain model is built upon several foundational principles.

- Business language comes first.
- Software reflects the business.

- Every concept has one meaning within one context.
- Contexts own their own models.
- Business behaviour belongs inside the domain.
- Technical concerns remain outside the domain.
- Domain models evolve continuously as understanding improves.
- Simplicity is preferred over theoretical completeness.

Every subsequent chapter expands one or more of these principles.

Document Lifecycle

MEG specifications evolve alongside the platform.

Each document progresses through the following lifecycle.

```
Draft

↓

Review

↓

Accepted

↓

Implemented

↓

Maintained

↓

Superseded (optional)
```

Accepted specifications become part of the canonical Mosaic architecture.

Historical versions SHOULD remain available for future reference.

Domain Evolution

Business understanding is expected to evolve.

Consequently, the domain model will evolve.

Changes affecting:

- bounded contexts
- ubiquitous language
- aggregates
- business ownership
- domain relationships

- core business concepts

SHOULD be accompanied by an Architectural Decision Record (ADR).

The evolution of the business model should remain deliberate and historically traceable.

Compliance

All business capabilities SHOULD comply with MEG-003.

Where deviation becomes necessary, the repository SHOULD document:

- the reason
- business motivation
- architectural implications
- migration strategy

Temporary deviations should eventually be removed.

Permanent deviations should generally result in updates to this specification.

Design Philosophy

MEG-003 intentionally favours:

- business-first thinking
- explicit ownership
- cohesive models
- bounded complexity
- expressive language
- evolutionary modelling

The domain model should become deeper as the platform matures.

It should never become more technical.

This follows the central ideas of Domain-Driven Design: focusing on the core domain, collaborating around a ubiquitous language and modelling within explicitly bounded contexts. [oai_citation:0‡Google Books](https://books.google.com/books/about/Domain_Driven_Design_Reference.html?id=ccRsBgAAQBAJ&utm_source=chatgpt.com)

Scope of Authority

MEG-003 governs business modelling.

It does **not** define:

- runtime execution

- event delivery
- scheduling
- transport protocols
- storage technologies
- deployment architecture

Those concerns belong to other MEG specifications.

Separating business modelling from technical implementation allows each to evolve independently.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`README.md`

Next File

`01-domain-philosophy.md`

Domain Philosophy

> *Software exists to solve business problems. The domain exists before the software, and should continue to make sense without it.*

Purpose

The purpose of Domain-Driven Design is not to introduce architectural patterns.

It is to build software that accurately reflects how the business understands itself.

Within Mosaic, the business is not:

- HTTP
- PostgreSQL
- Events
- Extensions
- Workers

The business is:

- Libraries
- Media
- Playback
- Metadata
- Collections
- Users
- Devices

Everything else exists to support those concepts.

This document establishes the philosophical foundation upon which every future domain model within Mosaic will be built.

Philosophy

Within Mosaic:

> ***The business defines the software. The software must never define the business.***

Engineers should resist the temptation to model implementation details.

Instead they should model the concepts that exist regardless of technology.

If Mosaic were rewritten tomorrow in another language, the domain should remain recognisable.

The implementation changes.

The business does not.

What Is A Domain?

A domain is the area of knowledge the software exists to support.

Examples include:

```
Banking

Healthcare

Retail
```

For Mosaic:

```
Media Management
```

The domain includes every concept required to organise, discover, consume and manage personal media.

Everything else supports that purpose.

Business Before Technology

One of the easiest mistakes engineers make is allowing technology to shape the domain.

Poor.

```
Database

↓

Tables

↓

Models

↓

Business
```

Preferred.

```
Business

↓

Concepts

↓

Domain Model

↓

Persistence
```

The database stores the domain.

It does not define it.

Likewise:

- APIs expose the domain.

- Events describe the domain.
- Extensions extend the domain.

None of them create it.

The Domain Is The Product

Within Mosaic, the domain **is** the product.

Users do not care that:

- PostgreSQL exists
- DuckDB exists
- Workers exist
- Event buses exist

They care about:

- their library
- watch history
- recommendations
- metadata
- collections

The architecture should therefore optimise for expressing those concepts clearly.

Deep Models

A domain model should become deeper over time.

Early understanding.

```
Media
```

Later understanding.

```
Movie

Series

Episode

Season

Collection
```

Even later.

```
Watched

In Progress

Abandoned

Favourite
```

```
Continue Watching
```

The model should evolve as understanding grows.

It should never be considered complete.

Eric Evans describes Domain-Driven Design as an iterative process where the model and the ubiquitous language continuously evolve together. [oai_citation:0‡Google Books](https://books.google.com/books/about/Domain_Driven_Design_Reference.html?id=ccRsBgAAQBAJ&utm_source=chatgpt.com)

The Cost Of Technical Thinking

Many systems accidentally model implementation rather than business.

Examples.

```
MediaDTO
```

```
MediaEntity
```

```
MediaRecord
```

```
MediaRow
```

None of these describe the business.

They describe implementation.

The domain should instead contain:

```
Media
```

Technical concerns belong elsewhere.

Software Should Read Like The Business

A business expert should recognise domain terminology.

For example.

```
PlaybackStarted
```

```
↓
```

```
Playback
```

```
↓
```

```
Watch Progress
```

```
↓
```

```
Library
```

These concepts make sense to a product owner.

Contrast with:

```
PlaybackManager
```

```
↓
```

```
MediaDTO
```

↓

```
PlaybackController
```

These names primarily communicate implementation.

The software should speak the language of the business wherever possible.

The Domain Is Not CRUD

Many applications begin with CRUD.

```
Create

Read

Update

Delete
```

CRUD describes data manipulation.

It does not describe business behaviour.

For example:

```
PlaybackCompleted
```

is far richer than:

```
Update Playback Record
```

Business behaviour should drive modelling.

CRUD naturally follows.

Not the other way around.

The Domain Is Not The Database

A common mistake is equating domain models with persistence models.

Example.

```
Database Table

↓

Go Struct

↓

Domain
```

Instead.

```
Domain Concept

↓

Persistence Model

↓

Database
```

Persistence exists to support the domain.

It should never constrain it unnecessarily.

The Domain Is Not The UI

Likewise:

User interfaces should project the domain.

They should not define it.

Example.

```
Continue Watching
```

is a business concept.

It may appear:

- on the Web
- on Android
- on iOS
- on TV

The UI presents it.

The domain owns it.

Business Behaviour Lives In The Domain

Business rules belong with business concepts.

Example.

Poor.

```
HTTP

↓

Validate Playback

↓

Repository

↓

Database
```

Preferred.

```
Playback

↓

Business Rules

↓
```

```
Repository
```

The domain should enforce its own rules.

Infrastructure merely supports them.

Model Behaviour

Many systems model data.

Mosaic models behaviour.

Instead of:

```
Media

↓

Fields
```

Think:

```
Media

↓

Can Be Imported

Can Be Played

Can Be Archived

Can Be Indexed
```

The domain is defined by what concepts **do**, not merely what they contain.

Business Concepts Have Owners

Every concept belongs somewhere.

Examples.

```
Playback

↓

Watch Progress

Library

↓

Collections

Metadata

↓

Artwork
```

Ownership should remain explicit.

If multiple capabilities own the same concept, the model requires refinement.

Evolution Is Expected

The first domain model is rarely correct.

Understanding improves through:

- implementation
- discussion
- user feedback
- operational experience

The domain should therefore evolve continuously.

Changing the domain model is not failure.

It is evidence that understanding has improved.

Simplicity

Domain models should remain as simple as possible.

Avoid modelling:

- hypothetical futures
- technical abstractions
- unnecessary hierarchies

Every concept should justify its existence through business value.

Complexity should emerge from the business.

Never from the architecture.

Mosaic Principles

Within Mosaic:

- Business concepts come before technical concepts.
- The domain defines the software.
- The domain is independent of infrastructure.
- Behaviour is more important than data.
- Models evolve continuously.
- Business ownership remains explicit.
- The software should speak the language of the business.
- Simplicity should always be preferred over unnecessary sophistication.

These principles guide every future modelling decision within the platform.

Relationship to MEG

The previous specifications established:

- how software is written
- how software executes

MEG-003 begins answering a different question.

> ***What exactly is the software modelling?***

Everything that follows builds upon this philosophical foundation.

The remaining chapters explain how that philosophy becomes an explicit, maintainable domain model.

Summary

The purpose of Domain-Driven Design is not to introduce Entities, Aggregates or Repositories.

Those are merely tools.

The true objective is much simpler.

> ***Create software that speaks the language of the business so naturally that the code itself becomes a model of the domain.***

When the software reflects the business rather than the technology, understanding becomes easier, change becomes cheaper and architecture becomes significantly more resilient.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`00-document-control.md`

Next File

`02-ubiquitous-language.md`

Ubiquitous Language

> *If two engineers use the same word to mean different things, the architecture has already begun to fail.*

Purpose

Software is built through communication.

Engineers discuss:

- requirements
- bugs
- architecture
- behaviour
- design

If every conversation uses different terminology, misunderstandings become inevitable.

Domain-Driven Design addresses this problem through a **Ubiquitous Language**.

Within Mosaic, every significant business concept should have:

- one name
- one meaning
- one owner

Everywhere.

This document defines how a common language is established and maintained throughout the Mosaic platform.

Philosophy

Within Mosaic:

> ***Every business concept should have one canonical name within its bounded context.***

The language used in:

- source code
- documentation
- ADRs
- architecture
- issues
- discussions

should all describe the domain identically.

Changing terminology changes understanding.

Consistency is therefore an architectural concern.

Why Language Matters

Imagine three engineers discussing the same concept.

Engineer A says:

```
Library
```

Engineer B says:

```
Collection
```

Engineer C says:

```
Catalogue
```

Do they mean the same thing?

Perhaps.

Perhaps not.

The conversation itself becomes ambiguous.

Ubiquitous Language eliminates that ambiguity.

One Concept, One Name

Every business concept should have exactly one canonical name.

Example.

```
Library
```

Never:

```
Catalogue
```

```
Media Collection
```

```
Content Repository
```

unless those genuinely represent different business concepts.

Synonyms create confusion.

Consistency creates understanding.

One Name, One Meaning

Likewise:

One word should never describe multiple concepts.

Poor.

```
Collection
```

Meaning:

- Movie Collection
- Database Collection
- Garbage Collection

Instead:

```
Collection

Table

Garbage Collection
```

Different concepts deserve different names.

Language Belongs To The Business

Business terminology should come from domain experts.

Not software engineers.

Example.

Users understand:

```
Continue Watching
```

They do not naturally understand:

```
Playback Resume Projection
```

The software should therefore adopt the business language.

Not invent its own.

Code Should Read Like Documentation

Good domain code should read naturally.

Example.

```
[go]
playback.Resume(user, media)
```

Rather than:

```
[go]
playback.ExecuteResumeOperation(...)
```

Or:

```
[go]
playback.ProcessPlaybackResumeHandler(...)
```

Business language naturally reduces unnecessary technical vocabulary.

Conversations Should Match Code

Suppose someone asks:

> *Why isn't Continue Watching updating?*

Engineers should immediately recognise:

`Continue Watching`

because:

- the package
- the domain
- the events
- the documentation

all use exactly the same phrase.

No translation should be required.

Business Before Technology

Avoid introducing technical terminology into the domain.

Poor.

`Projection`

`DTO`

`Entity Model`

`Persistence Object`

Preferred.

`Playback`

`Library`

`Metadata`

`Artwork`

`Recommendation`

Technical concepts belong in infrastructure.

Business concepts belong in the domain.

Context Matters

One important idea in Domain-Driven Design is that words may legitimately have different meanings in different bounded contexts.

Example.

`Collection`

Within Library:

A user-created grouping of media.

Within Database:

A MongoDB collection.

These meanings are acceptable because they belong to different contexts.

Language should remain consistent *within* a context.

Not necessarily globally.

This principle is central to Evans' concept of bounded contexts. ([books.google.com](https://books.google.com/books/about/Domain_Driven_Design_Reference.html?id=ccRsBgAAQBAJ&utm_source=chatgpt.com))

Domain Vocabulary

Every bounded context SHOULD maintain its own vocabulary.

Example.

Playback

`Play`

`Pause`

`Resume`

`Seek`

`Complete`

Library

`Import`

`Scan`

`Collection`

`Folder`

`Source`

Metadata

`Artwork`

`Synopsis`

`Cast`

`Episode`

`Season`

Each vocabulary should remain focused upon its own business concerns.

Events Use Ubiquitous Language

Event names should naturally reinforce the language.

Example.

`PlaybackStarted`

`PlaybackPaused`

`PlaybackCompleted`

Notice that every event reinforces:

`Playback`

Future contributors learn the language simply by reading the event stream.

APIs Use Ubiquitous Language

Public APIs should expose business terminology.

Example.

`/libraries`

Not:

`/catalogues`

unless those concepts genuinely differ.

The public API is part of the ubiquitous language.

Documentation Uses The Same Language

Architecture specifications.

README files.

ADRs.

Code comments.

All should use the same terminology.

Documentation should never introduce alternative names.

Doing so fragments understanding.

Refactoring Language

Language evolves.

Suppose the business decides:

`Watch History`

is better expressed as:

`Viewing History`

The change should occur consistently.

Not partially.

The ubiquitous language should evolve together.

Half-completed terminology changes create architectural confusion.

Avoid Abbreviations

Business concepts SHOULD rarely be abbreviated.

Poor.

`Recs`

Preferred.

`Recommendations`

Poor.

`Lib`

Preferred.

`Library`

Clarity outweighs brevity.

Avoid Technical Suffixes

Avoid names such as:

`MediaDTO`

`PlaybackEntity`

`LibraryRecord`

The domain should simply contain:

`Media`

`Playback`

`Library`

Infrastructure concerns remain elsewhere.

Language Review

Whenever introducing a new concept ask:

- Would a user understand this?
- Would a product owner use this term?
- Does another word already exist?

- Does this belong to the correct context?
- Does this conflict with existing terminology?

If uncertainty exists:

Improve the language before implementing the software.

Living Language

The ubiquitous language is never complete.

It evolves alongside:

- product understanding
- user feedback
- architectural knowledge
- domain discovery

Language refinement should therefore be viewed as continuous architectural improvement.

Mosaic Examples

Good.

```
Library

Collection

Continue Watching

Playback

Watch Progress

Metadata
```

Poor.

```
PlaybackManager

ContentDTO

LibraryServiceImpl

GenericProcessor
```

These names communicate implementation.

Not the business.

Mosaic Guidelines

Within Mosaic:

- Every business concept MUST have one canonical name.
- The same concept MUST NOT have multiple names within one bounded context.

- Business terminology MUST appear consistently across code and documentation.
- Event names MUST reinforce the ubiquitous language.
- APIs SHOULD expose business terminology.
- Technical implementation details MUST remain outside the domain language.
- The language SHOULD evolve alongside business understanding.

Relationship to the Domain

The ubiquitous language forms the foundation of every domain model.

Before defining:

- entities
- value objects
- aggregates
- services

engineers must first agree upon the language describing them.

Without a shared language, no shared model can exist.

The remaining chapters therefore build upon the vocabulary established here.

Summary

The ubiquitous language is one of the most powerful ideas within Domain-Driven Design.

It transforms software from:

> ***code describing implementation***

into

> ***code describing the business itself.***

When every engineer, architect, product owner and contributor uses the same language, communication becomes dramatically simpler.

Good architecture begins with good conversations.

Good conversations begin with a shared language.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`01-domain-philosophy.md`

Next File

`03-subdomains.md`

Subdomains

> *Not every part of the business deserves the same architectural attention. Great software invests where it creates the greatest business value.*

Purpose

The Mosaic platform encompasses many different business capabilities.

Examples include:

- Media Libraries
- Metadata
- Playback
- Authentication
- Search
- Recommendations
- Extensions
- Users
- Analytics

Treating every capability as equally important inevitably wastes engineering effort.

Domain-Driven Design instead encourages identifying different categories of business capability based upon their strategic value.

This document defines how Mosaic identifies, categorises and prioritises its subdomains.

Philosophy

Within Mosaic:

> ***Engineering effort should reflect business importance.***

Some capabilities define the identity of the platform.

Others simply support it.

Recognising this distinction allows engineering effort to be invested where it provides the greatest long-term value.

What Is A Subdomain?

A subdomain is an identifiable area of business responsibility within the overall domain.

The Mosaic domain is:

```
Media Management
```

Within that domain exist many subdomains.

Examples include:

```
Playback

Metadata

Libraries

Users

Search

Collections

Recommendations

Extensions
```

Each subdomain owns one coherent business capability.

Why Subdomains Exist

Without subdomains:

```
Media Platform

↓

One Huge Model
```

Everything becomes interconnected.

Understanding decreases.

Coupling increases.

Instead:

```
Media Platform

↓

Playback

Metadata

Library

Users

Extensions
```

Complexity becomes naturally partitioned.

Each area evolves independently.

Strategic Design

Domain-Driven Design distinguishes between different kinds of subdomains.

Within Mosaic these categories determine:

- engineering investment
- architectural ownership
- implementation quality
- extension opportunities

Not every capability deserves the same level of sophistication.

This classification is a key part of Evans' strategic design, helping teams focus effort on the areas that provide the greatest competitive advantage. ([books.google.com](https://books.google.com/books/about/Domain_Driven_Design_Reference.html?id=ccRsBgAAQBAJ&utm_source=chatgpt.com))

Core Domain

The Core Domain represents the primary reason Mosaic exists.

It defines the platform's competitive advantage.

Core Domains should receive:

- the strongest architecture
- the highest engineering quality
- the greatest testing investment
- the clearest business modelling

These capabilities should rarely be delegated to external systems.

Mosaic Core Domains

The following are currently considered Core Domains.

`Library`

Responsible for:

- organising media
- media ownership
- media identity
- user collections

`Playback`

Responsible for:

- media playback
- progress tracking
- playback state
- synchronisation

```
Metadata
```

Responsible for:

- metadata ownership
- artwork
- external providers
- metadata enrichment

These domains define the user experience.

They therefore define the platform.

Supporting Domains

Supporting Domains enable the Core Domain.

They remain important.

They simply do not define Mosaic's unique value.

Examples include:

```
Authentication

Notifications

Search

Import
```

Supporting domains should remain:

- cohesive
- independent
- replaceable

Engineering quality remains important.

Architectural complexity should remain proportional.

Generic Domains

Generic Domains solve common technical problems.

Examples include:

```
Logging

Metrics

Configuration

Blob Storage

Scheduling
```

These capabilities rarely differentiate Mosaic.

Whenever practical they should leverage:

- existing libraries
- established standards
- proven implementations

Engineering effort should remain focused upon the Core Domain instead.

Strategic Investment

Engineering effort should roughly follow this priority.

```
Core Domain

↓

Supporting Domain

↓

Generic Domain
```

Core Domains deserve custom solutions.

Generic Domains usually do not.

This principle prevents unnecessary engineering effort.

Domain Ownership

Every subdomain MUST have a clearly defined owner.

Example.

```
Playback

↓

Playback Team

Metadata

↓

Metadata Team
```

Ownership answers:

- who evolves the model
- who defines terminology
- who owns events
- who owns invariants

Shared ownership generally indicates unclear boundaries.

Independent Evolution

Subdomains should evolve independently.

Suppose:

`Recommendations`

changes dramatically.

`Playback`

should require little or no modification.

Boundaries should naturally isolate change.

This reduces maintenance costs.

Capability Alignment

Within Mosaic:

Every capability should align with exactly one primary subdomain.

Example.

`Metadata Extension`

↓

`Metadata Domain`

Not:

`Metadata`

`+`

`Playback`

`+`

`Users`

Cross-domain capabilities usually indicate unclear responsibilities.

Event Ownership

Subdomains own their own events.

Examples.

`Playback`

↓

`PlaybackStarted`

`Metadata`

↓

`MetadataFetched`

Other subdomains may subscribe.

They should never redefine ownership.

Storage Ownership

Likewise:

Every subdomain owns its own business state.

Poor.

```
Playback

↓

Modify Metadata Tables
```

Preferred.

```
Playback

↓

Publish Event

↓

Metadata Updates Itself
```

State ownership follows domain ownership.

Extension Alignment

Extensions should extend domains.

Not replace them.

Example.

```
Recommendation Extension

↓

Recommendation Domain
```

Extensions integrate naturally because the domain boundaries already exist.

The extension model therefore reinforces the domain model.

Domain Boundaries

Every subdomain should answer:

> ***What business capability do I own?***

Equally important:

> ***What business capability do I explicitly not own?***

Good boundaries define both.

Signs Of Poor Subdomains

The following usually indicate poor modelling.

- Constant cross-domain modifications.
- Shared business state.
- Shared terminology.
- Circular dependencies.
- Multiple owners.
- Frequent architectural disagreement.

These symptoms usually indicate boundaries require refinement.

Evolving Subdomains

Subdomains are expected to evolve.

Example.

Initially.

```
Metadata
```

Later.

```
Metadata

↓

Providers

↓

Artwork

↓

Translation
```

Understanding naturally increases.

Subdomains may split.

They should rarely merge.

Growing understanding generally produces more precise boundaries.

Example Mosaic Domain Map

```
Media Platform

■■■ Library
■
■■■ Playback
■
■■■ Metadata
■
■■■ Collections
■
■■■ Users
■
■■■ Authentication
■
■■■ Search
■
■■■ Recommendations
■
■■■ Extensions
```

This is **not** the implementation architecture.

It is the business architecture.

Implementation follows later.

Mosaic Guidelines

Within Mosaic:

- Every capability MUST belong to a subdomain.
- Every subdomain MUST own one business capability.
- Core Domains SHOULD receive the greatest engineering investment.
- Supporting Domains SHOULD enable Core Domains.
- Generic Domains SHOULD leverage existing solutions where practical.
- Business state MUST remain owned by its domain.
- Events MUST follow domain ownership.
- Domain boundaries SHOULD evolve as business understanding improves.

Relationship to MEG

Subdomains partition the business.

The next chapter introduces the mechanism that allows each subdomain to maintain its own independent model.

Those mechanisms are known as **Bounded Contexts**.

Subdomains answer:

> ***What business capabilities exist?***

Bounded Contexts answer:

> ***Where does each business model begin and end?***

Summary

Subdomains divide complexity into meaningful business capabilities.

They allow Mosaic to:

- invest engineering effort intelligently
- isolate change
- define ownership
- scale development
- grow through extensions

The platform becomes easier to evolve because every capability has a clearly defined place within the business.

Architecture becomes a reflection of the business itself.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`02-ubiquitous-language.md`

Next File

`04-bounded-contexts.md`

Bounded Contexts

> *A model only makes sense within the boundary in which its language is valid.*

Purpose

As software grows, different areas of the business naturally develop different models.

The word:

`Library`

may mean one thing to media management.

It may mean something entirely different to recommendation generation.

Attempting to force every part of a system to share a single universal model inevitably produces ambiguity, coupling and increasing complexity.

Bounded Contexts solve this problem.

They define the explicit boundaries within which a domain model remains valid.

This document establishes how Bounded Contexts are identified, designed and maintained throughout the Mosaic platform.

Philosophy

Within Mosaic:

> ***Every business model is correct within its own boundary.***

There is no single "global domain model."

Instead, Mosaic consists of many independently evolving models.

Each model exists only within the context that owns it.

Outside that context, translation may be required.

What Is A Bounded Context?

A Bounded Context is an explicit boundary around a domain model.

Inside the boundary:

- terminology has one meaning
- business rules are consistent
- ownership is unambiguous

Outside the boundary:

Those assumptions no longer apply.

The boundary therefore protects the integrity of the model.

Why Bounded Contexts Exist

Imagine modelling the entire Mosaic platform as one domain.

```
Media

↓

Playback

↓

Metadata

↓

Users

↓

Collections

↓

Recommendations

↓

Extensions
```

Eventually:

- concepts collide
- ownership becomes unclear
- changes ripple everywhere

Instead:

```
Playback

↓

Bounded Context

↓

Playback Model

Metadata

↓

Bounded Context

↓

Metadata Model
```

Each model evolves independently.

Context Defines Meaning

Consider the word:

```
Library
```

Within the Library Context it means:

> *A user's organised collection of media.*

Within a hypothetical Extension Marketplace it might mean:

> *A repository of installable capabilities.*

Both meanings are correct.

Because they exist in different contexts.

Context gives language meaning.

Without context, language becomes ambiguous.

One Model Per Context

Every Bounded Context owns exactly one canonical domain model.

Example.

```
Playback Context

↓

Playback

Session

Progress

Resume Point
```

Those concepts belong exclusively to Playback.

Metadata should never redefine them.

Context Ownership

Every Bounded Context has exactly one owner.

Examples.

```
Playback Context

↓

Playback Capability

Metadata Context

↓

Metadata Capability
```

Ownership answers:

- who defines terminology
- who owns business rules
- who owns persistence
- who publishes events

Shared ownership usually indicates poorly defined boundaries.

Independence

Contexts should evolve independently.

Suppose:

`Metadata`

changes provider strategy.

Playback should remain unaffected.

The boundary prevents implementation details from leaking between contexts.

Reducing this coupling is one of the principal goals of bounded contexts in Domain-Driven Design. ([martinfowler.com](https://martinfowler.com/bliki/BoundedContext.html))

Context Boundaries

Every Bounded Context owns:

- language
- business rules
- entities
- value objects
- aggregates
- repositories
- domain events

Everything inside the boundary belongs to that context.

Everything outside belongs elsewhere.

Context Interaction

Contexts communicate through well-defined contracts.

Within Mosaic those contracts are typically:

- domain events

- public interfaces
- extension APIs

Contexts SHOULD NOT communicate through:

- shared databases
- shared models
- direct knowledge of internal state

Boundaries should remain explicit.

Shared Database Does Not Mean Shared Context

Two contexts may use the same physical database.

They do **not** therefore share a model.

Poor.

```
Playback

↓

Directly Updates Metadata Tables
```

Preferred.

```
Playback

↓

PlaybackCompleted

↓

Metadata Reacts
```

Database topology should never define architectural boundaries.

Context Translation

Sometimes concepts must cross boundaries.

Example.

```
Playback

↓

PlaybackCompleted

↓

Recommendations
```

Recommendations should interpret the event using its own model.

It should not adopt Playback's internal concepts.

Translation protects both contexts from unnecessary coupling.

Context Autonomy

Each context should be capable of evolving independently.

Examples.

```
Metadata

↓

New Provider
```

Should not require:

```
Playback Changes
```

Likewise:

```
Recommendations

↓

Machine Learning

↓

Playback Unchanged
```

Autonomy enables long-term evolution.

Context Size

Contexts should remain cohesive.

Poor.

```
Media

↓

Everything
```

Better.

```
Playback

Metadata

Library
```

Too-large contexts become mini-monoliths.

Too-small contexts create unnecessary complexity.

The boundary should reflect genuine business responsibility.

Context Language

Within a Bounded Context:

Every concept has exactly one meaning.

Example.

Playback.

```
Resume Point
```

Metadata should not define:

```
Resume Point
```

unless it genuinely owns the concept.

Language follows ownership.

Domain Events Cross Boundaries

Events are the preferred mechanism for communicating between contexts.

Example.

```
Playback Context

↓

PlaybackCompleted

↓

Recommendations Context

↓

RecommendationUpdated
```

Neither context understands the other's internal model.

Events become the contract.

Extensions And Contexts

Extensions should align with one primary Bounded Context.

Example.

```
TMDB Extension

↓

Metadata Context

Infuse Extension

↓

Playback Context
```

Extensions should not simultaneously own multiple unrelated business models.

If they do, their responsibilities should be reconsidered.

Context Boundaries Are Stronger Than Packages

A package is a code organisation mechanism.

A Bounded Context is a business boundary.

One Bounded Context may contain many packages.

Those packages should all describe the same business model.

Implementation follows the context.

Not the other way around.

Signs Of A Healthy Context

Healthy contexts exhibit:

- clear ownership
- cohesive terminology
- independent evolution
- explicit contracts
- minimal coupling
- stable boundaries

Engineers should be able to explain the responsibility of a context in one sentence.

Signs Of A Weak Context

The following usually indicate boundary problems.

- shared entities
- shared repositories
- direct database access
- duplicated ownership
- circular dependencies
- constantly changing terminology

When these symptoms appear, revisit the business model before changing the implementation.

Example Mosaic Context Map

```
Library Context

↓

MediaImported

↓

Metadata Context

↓

MetadataFetched

↓

Playback Context

↓

PlaybackStarted

↓

Recommendations Context

↓

RecommendationsUpdated
```

Notice:

Every context owns its own model.

Communication occurs through events.

Not shared objects.

Mosaic Guidelines

Within Mosaic:

- Every domain model MUST belong to one Bounded Context.
- Every Bounded Context MUST have one owner.
- Language MUST remain consistent within the context.
- Business rules MUST remain inside the owning context.
- Contexts SHOULD communicate through events.
- Shared models SHOULD be avoided.
- Shared databases MUST NOT imply shared ownership.
- Context boundaries SHOULD evolve only through deliberate architectural review.

Relationship to MEG

Subdomains identify:

> ***What business capabilities exist?***

Bounded Contexts define:

> ***Where each business model is valid.***

The next chapter introduces **Context Maps**, which describe how those independent contexts relate to one another across the Mosaic platform.

Summary

Bounded Contexts are one of the most important concepts in Domain-Driven Design.

They protect the integrity of business models by ensuring that:

- language remains consistent
- ownership remains explicit
- change remains isolated
- architecture remains understandable

Within Mosaic, every capability grows safely because every domain model has a clearly defined boundary.

Without boundaries, there is only one large model.

With boundaries, there is a platform.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`03-subdomains.md`

Next File

`05-context-maps.md`

Context Maps

> *Bounded Contexts define where models are valid. Context Maps define how those models collaborate.*

Purpose

A Mosaic platform consists of many independent Bounded Contexts.

Examples include:

- Library
- Metadata
- Playback
- Recommendations
- Authentication
- Extensions

Although each context evolves independently, they rarely exist in complete isolation.

Business capabilities inevitably collaborate.

Without explicitly modelling those relationships, dependencies gradually become informal, undocumented and increasingly difficult to maintain.

Context Maps make those relationships explicit.

Philosophy

Within Mosaic:

> ***Contexts collaborate through defined relationships. They never depend upon accidental ones.***

Every relationship between two Bounded Contexts should be:

- intentional
- documented
- understandable
- independently evolvable

No context should become tightly coupled simply because implementation made it convenient.

What Is A Context Map?

A Context Map describes the relationships between Bounded Contexts.

It answers questions such as:

- Which context owns this concept?

- Which context depends upon another?
- How do they communicate?
- Who translates between models?
- Where are architectural boundaries?

Context Maps describe architecture.

Not implementation.

Why Context Maps Matter

Without explicit relationships:

```
Playback
↓
Metadata
↓
Library
↓
Recommendations
↓
Users
```

Dependencies slowly become:

- circular
- implicit
- undocumented

Eventually:

Nobody understands why changing one context affects another.

Context Maps prevent this.

Context Relationships

Every relationship should answer:

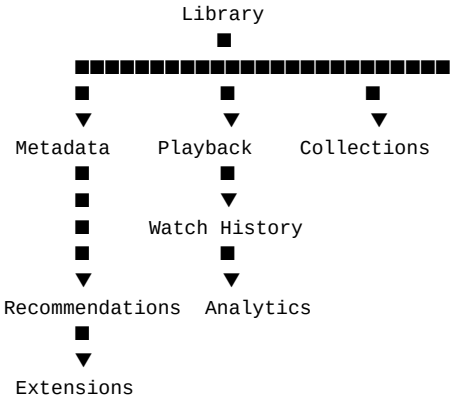
```
Who owns the model?
↓
Who consumes the model?
↓
How is information exchanged?
```

Ownership should always remain obvious.

Consumers should never redefine another context's business concepts.

Mosaic Context Map

At a high level, the Mosaic platform resembles:



This diagram represents business relationships.

Not package dependencies.

Direction Of Knowledge

Knowledge should always flow in one direction.

Example.

```
Library
```

↓

```
MediaImported
```

↓

```
Metadata
```

Metadata learns:

A media item was imported.

Library does **not** learn:

How metadata is fetched.

Dependencies remain one directional.

Relationship Types

Domain-Driven Design defines several relationship patterns.

Within Mosaic, the following are recognised.

- Partnership

- Customer / Supplier
- Conformist
- Open Host Service
- Published Language
- Anti-Corruption Layer

Not every relationship appears within every project.

Understanding them allows engineers to choose the most appropriate collaboration model.

Published Language

The preferred relationship within Mosaic.

```
Playback

↓

Domain Events

↓

Recommendations
```

Playback publishes a stable business language.

Recommendations consume it.

Neither context understands the other's internal implementation.

Published Language naturally complements the event-driven runtime established in MEG-002.

Open Host Service

Some contexts expose stable public interfaces.

Example.

```
Metadata

↓

Metadata API

↓

Extensions
```

Extensions interact through published contracts.

They do not depend upon internal implementation.

The interface remains stable even if the implementation evolves.

Anti-Corruption Layer

External systems frequently use different models.

Example.

```
TMDB

↓

Anti-Corruption Layer

↓

Metadata Context
```

The translation layer prevents external terminology from leaking into the Mosaic domain.

Examples include:

- Jellyfin
- Plex
- Stremio
- TMDB
- AniList
- Trakt

Every external integration SHOULD terminate at an Anti-Corruption Layer.

This is one of the defining tactical patterns in Domain-Driven Design for protecting an internal model from external concepts. ([martinfowler.com](https://martinfowler.com/bliki/AntiCorruptionLayer.html))

Conformist

Sometimes the cost of translation outweighs the benefit.

Example.

```
OpenTelemetry

↓

Observability Context
```

Rather than inventing new terminology, Mosaic simply adopts the external standard.

Conformist relationships should remain rare.

The Core Domain should never become conformist to an external product.

Customer / Supplier

One context depends upon another.

Example.

```
Playback
```

↓

```
PlaybackCompleted
```

↓

```
Recommendations
```

Playback supplies business facts.

Recommendations consumes them.

Playback should remain unaware of Recommendations.

Recommendations depends upon Playback.

Not the reverse.

Partnership

Occasionally two contexts evolve together.

Example.

```
Playback
```

↔

```
Watch History
```

Both contexts share significant business knowledge.

Partnerships should be introduced cautiously.

They naturally increase coupling.

Shared Kernel

Within Mosaic:

Shared Kernels SHOULD generally be avoided.

A Shared Kernel creates:

```
Context A
```

↓

```
Shared Model
```

↓

```
Context B
```

Both contexts now evolve together.

This increases coupling.

Instead, prefer:

- events
- published contracts

- anti-corruption layers

Shared Kernels should remain exceptional.

Context Translation

Translation occurs at boundaries.

Example.

```
Jellyfin

↓

Jellyfin Adapter

↓

Library Context
```

Library never understands:

- Jellyfin models
- Jellyfin terminology
- Jellyfin identifiers

Adapters translate.

Contexts remain pure.

Event Relationships

Events naturally define Context Maps.

Example.

```
Library

↓

MediaImported

↓

Metadata

↓

MetadataFetched

↓

Recommendations
```

Each event communicates:

- ownership
- dependency direction
- collaboration

The event graph effectively becomes the Context Map.

Extension Relationships

Extensions participate as independent contexts.

Example.

```
Core Metadata

↓

MetadataFetched

↓

Anime Extension
```

The extension consumes published language.

It never modifies Core.

This allows extensions to remain isolated while participating fully within the platform.

Avoid Bidirectional Dependencies

Poor.

```
Playback

↓

Metadata

↓

Playback
```

Bidirectional dependencies rapidly increase complexity.

Instead.

```
Playback

↓

PlaybackCompleted

↓

Metadata
```

The dependency direction remains clear.

Context Independence

Every context should be removable.

Ask:

> ***If this context disappeared tomorrow, what would break?***

Ideally:

Only published contracts fail.

Other contexts remain internally consistent.

This is a useful test of architectural coupling.

Evolution

Context relationships evolve.

Initially.

```
Library

↓

Metadata
```

Later.

```
Library

↓

Metadata

↓

Recommendations

↓

Collections

↓

Statistics
```

Growth should occur through new relationships.

Existing relationships should remain stable wherever practical.

Context Ownership

Every Context Map SHOULD identify:

- owning context
- consuming context
- communication mechanism
- translation boundary

Ambiguous ownership is an architectural smell.

Anti-Patterns

The following practices are prohibited.

Shared Domain Models

```
Playback

↓

Shared Media Object

↓

Metadata
```

Contexts should own their own models.

Shared Database Ownership

Multiple contexts directly modifying the same business tables.

Circular Relationships

```
Playback

↓

Metadata

↓

Playback
```

Leaking External Models

Allowing:

```
TMDB

↓

Metadata Context
```

without translation.

Hidden Dependencies

Relationships that exist only in implementation.

Every significant relationship should appear within the Context Map.

Mosaic Guidelines

Within Mosaic:

- Every Bounded Context SHOULD appear within a Context Map.
- Relationships MUST have explicit ownership.
- Published Language SHOULD be the preferred collaboration model.
- Anti-Corruption Layers SHOULD protect Core Domains.
- Shared Kernels SHOULD be avoided.
- Event relationships SHOULD define dependency direction.
- Contexts SHOULD remain independently evolvable.
- Context Maps SHOULD evolve alongside the business.

Relationship to MEG

Bounded Contexts establish:

> ***Where models are valid.***

Context Maps establish:

> ***How those models collaborate.***

The next chapter begins exploring the building blocks that exist *inside* those contexts, beginning with **Entities**.

Summary

Context Maps transform a collection of isolated Bounded Contexts into a coherent platform architecture.

They make dependencies:

- explicit
- understandable
- intentional

Within Mosaic, they also reinforce one of the platform's most important architectural goals:

> ***Capabilities collaborate without becoming coupled.***

When every relationship is explicit, the platform can continue growing without sacrificing architectural clarity.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`04-bounded-contexts.md`

Next File

`06-entities.md`

Entities

> *An Entity is defined by its identity. Its attributes may change throughout its lifetime, but it remains the same conceptual thing.*

Purpose

Many business concepts evolve over time.

For example:

- media is renamed
- artwork changes
- playback progresses
- users update preferences
- metadata improves

Although their attributes change, the business still considers them to be the same thing.

Domain-Driven Design models these concepts as **Entities**.

This document defines how Entities should be identified, designed and maintained throughout the Mosaic platform.

Philosophy

Within Mosaic:

> ***Identity defines an Entity. State merely describes it.***

An Entity exists because the business recognises it as something with a continuous lifecycle.

Changing its attributes does not create a new Entity.

Its identity remains constant.

What Is An Entity?

An Entity is a business object whose identity remains stable throughout its lifetime.

Examples include:

```
Media

User

Collection

Playback Session
```

Each has:

- identity

- lifecycle
- behaviour
- mutable state

The Entity continues to exist even as that state evolves.

Identity

Identity is the defining characteristic of an Entity.

Example.

```
Media

↓

Media ID

↓

Title Changes

↓

Artwork Changes

↓

Metadata Changes

↓

Still The Same Media
```

Everything except identity may change.

Identity must remain stable.

Identity Is Not Storage

Entity identity belongs to the business.

Not the database.

Poor.

```
Database Row

↓

Primary Key

↓

Entity
```

Preferred.

```
Business Concept

↓
```

```
Business Identity
↓
Persistence
```

The database stores identity.

It does not create it.

Business Identity

Identity should represent something meaningful to the business.

Examples.

```
MediaID
UserID
CollectionID
```

Avoid exposing infrastructure concepts such as:

```
RowNumber
AutoIncrement
```

The business should not depend upon database implementation.

Lifecycle

Every Entity has a lifecycle.

Example.

```
Media
↓
Imported
↓
Metadata Updated
↓
Played
↓
Archived
↓
Deleted
```

The Entity remains the same throughout.

Only its state changes.

Mutable State

Entities naturally contain mutable state.

Example.

```
Playback

↓

Position

↓

Watched Percentage

↓

Completed
```

Changing these values does not create a new Playback Entity.

The identity remains constant.

Behaviour

Entities are not merely data containers.

They should own business behaviour.

Poor.

```
[go]
media.Title = title
```

Better.

```
[go]
media.Rename(title)
```

Or:

```
[go]
playback.Resume(position)
```

Business rules belong inside the Entity.

Not inside services manipulating it.

This aligns with Domain-Driven Design's emphasis on rich domain models that encapsulate behaviour alongside state. ([martinfowler.com](https://martinfowler.com/bliki/AnemicDomainModel.html))

Invariants

Entities are responsible for protecting their own invariants.

Example.

A Playback Entity should never allow:

```
Progress

↓
```

```
120%
```

Or:

```
Negative Duration
```

Invalid state should be impossible through the Entity's public behaviour.

Future chapters discuss invariants in greater detail.

Entity Ownership

Every Entity belongs to exactly one Bounded Context.

Example.

```
Playback Context

↓

Playback Entity

Metadata Context

↓

Metadata Entity
```

Entities should never belong simultaneously to multiple contexts.

If another context requires similar concepts, it should model its own Entity.

Entity References

Entities should reference other Entities by identity.

Preferred.

```
Playback

↓

MediaID
```

Not.

```
Playback

↓

Entire Media Object
```

Referencing identities rather than object graphs reduces coupling between aggregates and bounded contexts.

Equality

Entities are equal if their identities are equal.

Example.

```
Media
```

↓

```
ID = 123
```

Even if:

- title differs
- artwork differs
- metadata differs

the business still considers them the same Entity.

State does not determine identity.

Identity determines identity.

Creation

Entities should be created in a valid state.

Poor.

```
[go]
media := &Media{}
```

Later.

```
Populate Fields
```

↓

```
Validate
```

Preferred.

```
[go]
media := NewMedia(...)
```

The constructor establishes valid business state immediately.

Invalid Entities should never exist.

Persistence

Repositories persist Entities.

Entities should remain unaware of:

- SQL
- PostgreSQL
- DuckDB
- HTTP
- JSON

Persistence is infrastructure.

Identity and behaviour belong to the domain.

Events

Entities frequently publish Domain Events.

Example.

```
Playback

↓

Complete()

↓

PlaybackCompleted
```

The Entity owns the business transition.

Publishing the corresponding Domain Event naturally follows.

This keeps business behaviour and business facts closely aligned.

Avoid Anemic Entities

Poor.

```
[go]
type Media struct {
    Title string
}
```

Followed by:

```
MediaService

↓

Rename()

Validate()

Archive()

Delete()
```

The Entity has become little more than a database record.

Instead:

```
[go]
media.Rename()

media.Archive()

media.Restore()
```

Behaviour belongs with the concept that owns it.

Entity Size

Entities should remain cohesive.

If an Entity begins owning:

- playback
- metadata
- recommendations
- collections
- analytics

it has probably become multiple business concepts.

Split behaviour according to business responsibility.

Entity Relationships

Entities collaborate.

They should not become deeply nested object graphs.

Example.

```
Collection

↓

MediaID

↓

Media Retrieved Through Repository
```

Avoid loading the entire domain into memory unnecessarily.

Relationships should remain explicit.

Entity Evolution

Entity behaviour should evolve as understanding improves.

Initially.

```
Media

↓

Title
```

Later.

```
Media

↓

Rename()

Archive()

Restore()

Move()
```

```
ChangeArtwork()
```

Behaviour grows alongside business understanding.

Entities should become richer.

Not larger.

What Is Not An Entity?

The following are usually **not** Entities.

- Duration
- Resolution
- Rating
- Language
- Genre

These have no meaningful identity.

They are better modelled as **Value Objects**.

The next chapter explores this distinction.

Mosaic Examples

Examples of Entities within Mosaic include:

```
Media
```

```
User
```

```
Collection
```

```
PlaybackSession
```

```
Library
```

Each represents a business concept with:

- stable identity
- evolving state
- business behaviour

Anti-Patterns

The following practices are prohibited.

Anemic Entities

Entities containing only fields.

Mutable Identity

Changing an Entity's identity after creation.

Infrastructure Dependencies

Entities importing:

- SQL
- HTTP
- JSON
- Logging

Shared Ownership

Entities belonging to multiple Bounded Contexts.

Behaviour In Services

Business behaviour implemented entirely outside the Entity.

Mosaic Guidelines

Within Mosaic:

- Every Entity MUST have stable identity.
- Identity MUST represent business concepts.
- Entities SHOULD own business behaviour.
- Entities MUST enforce their own invariants.
- Entities MUST belong to one Bounded Context.
- Equality MUST be based upon identity.
- Entities SHOULD reference other Entities by identity.
- Entities MUST remain independent of infrastructure.

Relationship to MEG

Entities are the first building block inside a Bounded Context.

However, not every business concept has identity.

Many concepts are defined entirely by their value.

The next chapter introduces **Value Objects**, which complement Entities by modelling concepts that have meaning without requiring identity.

Together, Entities and Value Objects form the foundation of every rich domain model.

Summary

Entities represent the enduring concepts of the business.

They are recognised not because of what they contain, but because of **who they are**.

Within Mosaic, Entities:

- own identity
- own behaviour
- protect invariants
- evolve over time

By placing behaviour alongside business concepts, the domain model becomes more expressive, more maintainable and significantly closer to the language of the business itself.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`05-context-maps.md`

Next File

`07-value-objects.md`

Value Objects

> *A Value Object is defined entirely by what it is, not by who it is.*

Purpose

Not every business concept requires identity.

Many concepts exist purely because of the information they represent.

Examples include:

- Duration
- Resolution
- Language
- Rating
- Position
- File Size

The business does not distinguish between two identical durations.

Nor does it care whether one language object was created before another.

These concepts are modelled as **Value Objects**.

This document defines how Value Objects should be designed and used throughout the Mosaic platform.

Philosophy

Within Mosaic:

> ***If identity is irrelevant, model the concept as a Value Object.***

Value Objects exist because of their value.

Two Value Objects containing identical values represent exactly the same business concept.

Unlike Entities, they possess no lifecycle or identity.

What Is A Value Object?

A Value Object is a business concept whose identity is irrelevant.

Examples include:

```
Duration

Resolution

Aspect Ratio

Language

Watch Position
```

Each represents a meaningful business concept.

None require identity.

Value Defines Equality

Unlike Entities:

Value Objects are equal when all of their values are equal.

Example.

```
Duration

↓

01:30:00
```

equals

```
Duration

↓

01:30:00
```

There is no business distinction between them.

Creating another identical Value Object creates the same concept.

No Identity

Value Objects MUST NOT possess business identity.

Poor.

```
ResolutionID

LanguageID

DurationID
```

These identifiers communicate nothing useful.

The value itself already defines the concept.

Immutability

Value Objects SHOULD be immutable.

Changing:

```
Duration

↓

90 Minutes

↓

95 Minutes
```

does not modify the existing Value Object.

Instead:

```
Old Value Object

↓

New Value Object
```

Immutability greatly simplifies reasoning, testing and concurrent execution.

It also aligns naturally with Go's preference for immutable value semantics where practical.

Behaviour

Value Objects may contain behaviour.

Example.

```
[go]
duration.Add(other)

[go]
position.Progress(total)

[go]
resolution.IsHDR()
```

Behaviour should naturally belong with the concept.

Avoid reducing Value Objects to passive data containers.

Rich Concepts

Prefer:

```
Watch Position
```

rather than:

```
int64
```

Prefer:

```
Media Duration
```

rather than:

```
time.Duration
```

The standard library type may still be used internally.

However, wrapping important business concepts inside explicit Value Objects communicates intent more clearly.

Primitive Obsession

One of the most common modelling mistakes is representing business concepts using primitive types.

Poor.

```
[go]
type Media struct {
    Duration int
}
```

Questions immediately arise.

Duration:

- Seconds?
- Milliseconds?
- Frames?

Instead.

```
[go]
type Media struct {
    Duration Duration
}
```

The domain becomes significantly clearer.

Martin Fowler refers to overuse of primitive types in place of richer domain concepts as **Primitive Obsession**, a common code smell. ([martinfowler.com](https://martinfowler.com/bliki/ValueObject.html))

Validation

Value Objects SHOULD validate themselves during construction.

Example.

```
[go]
duration := NewDuration(seconds)
```

Validation may include:

- non-negative values
- valid ranges
- supported formats

Invalid Value Objects should never exist.

Constructors should enforce correctness immediately.

Side Effects

Value Objects MUST NOT have side effects.

They should never:

- publish events
- persist state
- call external systems
- modify repositories

They simply represent business concepts.

Their behaviour should remain deterministic.

Ownership

Value Objects belong to the Entity or Aggregate that owns them.

Example.

```
Media
```

↓

```
Duration
```

↓

```
Resolution
```

↓

```
Language
```

The Value Objects have no independent lifecycle.

If the Media Entity disappears, so do its associated Value Objects.

Persistence

Repositories persist Value Objects as part of their owning Entity or Aggregate.

Value Objects should never have independent repositories.

Poor.

```
ResolutionRepository
```

Unless Resolution has become a business concept with identity, this repository should not exist.

Reuse

Value Objects should be reused whenever the same business concept appears.

Example.

```
Duration
```

should represent duration consistently throughout:

- Playback
- Metadata
- Library
- Video Analysis

The ubiquitous language should remain consistent.

Value Object Composition

Value Objects may contain other Value Objects.

Example.

```
Video Format

↓

Resolution

↓

Frame Rate

↓

Aspect Ratio
```

This produces richer, more expressive models without introducing identity.

Entities Use Value Objects

Entities should compose Value Objects.

Example.

```
Media

■■■ MediaID
■■■ Duration
■■■ Resolution
■■■ Language
■■■ Artwork
```

Notice:

Only one concept possesses identity.

Everything else represents value.

This naturally produces smaller, more focused Entities.

Avoid Shared Mutable State

Because Value Objects are immutable, they may safely be shared.

Example.

```
English Language

↓

Media A

↓

Media B

↓

Media C
```

Sharing immutable values introduces no coupling.

Immutability greatly simplifies concurrent systems.

Avoid Generic Types

Avoid modelling business concepts using:

```
string
```

```
int
```

```
bool
```

where richer concepts exist.

Instead of:

```
[go]
Rating int
```

Prefer:

```
[go]
Rating Rating
```

The additional type communicates business meaning.

The compiler also prevents accidental misuse.

Business Language

Value Objects should reinforce the ubiquitous language.

Good.

```
WatchProgress
```

```
PlaybackPosition
```

```
ArtworkType
```

Poor.

```
ProgressDTO
```

```
PositionValue
```

```
ImageInfo
```

Names should communicate business concepts.

Not implementation.

Evolution

Value Objects often become richer over time.

Initially.

```
Resolution
```

↓

```
Width
```

```
Height
```

Later.

```
Resolution
```

↓

```
Width
```

```
Height
```

```
Aspect Ratio
```

```
Orientation
```

```
HDR Support
```

Business understanding evolves.

Value Objects should evolve alongside it.

What Is Not A Value Object?

The following are usually **not** Value Objects.

```
Media
```

```
User
```

```
Collection
```

```
Playback Session
```

These possess identity.

They are therefore Entities.

Mosaic Examples

Examples of Value Objects within Mosaic include:

```
Duration
```

```
Resolution
```

```
AspectRatio
```

```
Language
```

```
PlaybackPosition
```

```
MediaRating
```

```
ArtworkType
```

```
FileHash
```

Each is recognised because of its value.

Not because of an independent identity.

Anti-Patterns

The following practices are prohibited.

Mutable Value Objects

Changing the internal state of an existing Value Object.

Identity

Assigning identifiers to Value Objects.

Primitive Obsession

Representing business concepts entirely using primitive types.

Infrastructure Dependencies

Importing:

- SQL
- HTTP
- Logging
- Runtime

into Value Objects.

Side Effects

Publishing events or modifying repositories.

Independent Persistence

Persisting Value Objects independently from their owning Entity.

Mosaic Guidelines

Within Mosaic:

- Value Objects MUST be defined by value.
- Value Objects SHOULD be immutable.
- Value Objects MUST NOT possess identity.
- Value Objects SHOULD validate themselves during construction.
- Value Objects MAY contain behaviour.

- Value Objects MUST remain infrastructure independent.
- Entities SHOULD compose Value Objects.
- Business concepts SHOULD be preferred over primitive types.

Relationship to MEG

Entities answer:

> ***Who is this?***

Value Objects answer:

> ***What is this?***

Together they form the fundamental vocabulary of every rich domain model.

The next chapter introduces **Aggregates**, which define how multiple Entities and Value Objects collaborate while preserving business consistency.

Summary

Value Objects are one of the simplest yet most powerful modelling tools within Domain-Driven Design.

By representing business concepts through immutable values rather than primitive types, the Mosaic domain becomes:

- more expressive
- more type-safe
- easier to understand
- easier to test
- naturally thread-safe

Most importantly, the software begins speaking the language of the business rather than the language of the implementation.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`06-entities.md`

Next File

08-aggregates.md

Aggregates

> *An Aggregate is not a collection of objects. It is the consistency boundary of the business.*

Purpose

Business rules rarely apply to a single Entity in isolation.

For example:

- a Library owns many Media items
- a Collection owns many references to Media
- a Playback Session owns Watch Progress
- a User owns Preferences

These related concepts must remain internally consistent.

Domain-Driven Design models these consistency boundaries as **Aggregates**.

This document defines how Aggregates should be identified, designed and maintained throughout the Mosaic platform.

Philosophy

Within Mosaic:

> ***Aggregates own business consistency.***

An Aggregate exists to protect business invariants.

It is not simply a convenient grouping of related objects.

Every Aggregate should answer one question.

> ***Which business rules must always remain true together?***

What Is An Aggregate?

An Aggregate is a cluster of related domain objects treated as a single consistency boundary.

It consists of:

- one Aggregate Root
- zero or more Entities
- zero or more Value Objects

Everything inside the Aggregate participates in maintaining one coherent business concept.

Why Aggregates Exist

Suppose a Playback Session contains:

- current position
- watched duration
- completion state

These values cannot evolve independently.

If playback reaches 100%, the session must also become completed.

Without an Aggregate:

```
Playback

↓

Position Updated

↓

Completion Forgotten
```

The business becomes inconsistent.

Aggregates prevent this.

Consistency Boundary

The Aggregate defines the boundary within which business consistency is guaranteed.

```
Aggregate

↓

Business Rules

↓

Always Consistent
```

Outside the Aggregate:

Only eventual consistency should generally be assumed.

This distinction aligns naturally with Mosaic's event-driven runtime.

Aggregate Structure

Every Aggregate follows the same conceptual model.

```
Aggregate Root

■■■ Entity

■■■ Entity

■■■ Value Object

■■■ Value Object
```

The Aggregate Root controls every modification.

Nothing else may mutate Aggregate state directly.

Aggregate Size

Aggregates SHOULD remain small.

Poor.

```
Library

↓

Everything
```

Better.

```
Library

Collection

Playback
```

Each Aggregate should represent one coherent business concept.

Large Aggregates reduce concurrency, increase coupling and become increasingly difficult to evolve.

Evans emphasises keeping Aggregates as small as practical while still protecting business invariants. ([dddcommunity.org](https://dddcommunity.org/wp-content/uploads/files/pdf_articles/Vernon_2011_1.pdf))

One Transaction

Business consistency is guaranteed only inside one Aggregate.

Example.

```
Playback Session

↓

Update Position

↓

Mark Complete

↓

Commit
```

Everything succeeds.

Or nothing succeeds.

Across Aggregates:

Consistency is eventual.

Aggregate Boundaries

Aggregates should be identified through business rules.

Ask:

> ***Which information must always change together?***

Not:

> ***Which objects reference each other?***

Relationships alone do not justify an Aggregate.

Consistency does.

Aggregate Ownership

Every Aggregate belongs to exactly one Bounded Context.

Example.

```
Playback Context

↓

Playback Aggregate

Library Context

↓

Library Aggregate
```

Aggregates must never span multiple Bounded Contexts.

Context boundaries remain stronger than Aggregate boundaries.

Aggregate Behaviour

Business behaviour belongs inside the Aggregate.

Poor.

```
PlaybackService

↓

Update Position

↓

Validate

↓

Complete
```

Preferred.

```
Playback

↓
```

```
Advance()
↓
Complete()
```

The Aggregate enforces its own rules.

Services coordinate.

Aggregates decide.

Aggregate State

Internal state should never become invalid.

Example.

Poor.

```
Playback
↓
Progress = 120%
```

Preferred.

```
Playback.Advance()
↓
Validation
↓
Progress = 100%
```

Invalid state should never be observable.

Aggregate References

Aggregates SHOULD reference other Aggregates by identity.

Example.

```
Collection
↓
MediaID
```

Not.

```
Collection
↓
Entire Media Aggregate
```

Identity references reduce coupling.

Loading large object graphs should be avoided.

Aggregate Communication

Aggregates communicate through:

- Domain Events
- identities
- repositories

They SHOULD NOT directly modify another Aggregate.

Example.

Poor.

```
Playback

↓

Modify User Aggregate
```

Preferred.

```
PlaybackCompleted

↓

User Context Reacts
```

The runtime coordinates.

Aggregates remain autonomous.

Aggregate Lifetime

The Aggregate Root owns the lifetime of everything inside it.

Example.

```
Collection

↓

Collection Item

↓

Artwork Preference
```

Removing the Aggregate removes its internal concepts.

Internal Entities should not outlive their owning Aggregate.

Transactions

One transaction should modify one Aggregate.

Poor.

```
Playback

+
```

```
User
+
Metadata
↓
Single Transaction
```

Preferred.

```
Playback
↓
Commit
↓
PlaybackCompleted
↓
Other Aggregates React
```

This naturally complements the Event-Driven Runtime defined in MEG-002.

Aggregate Invariants

Aggregates enforce business invariants.

Examples.

Playback.

- Progress cannot exceed duration.
- Completion requires reaching the end.
- Resume position cannot be negative.

Collection.

- Duplicate media references prohibited.
- Collection name required.

Business rules belong inside the Aggregate.

Not inside controllers or repositories.

Domain Events

Aggregates are the primary source of Domain Events.

Example.

```
Playback Aggregate
↓
Complete()
```

```
↓
PlaybackCompleted
```

Business events originate from business behaviour.

Not infrastructure.

Persistence

Repositories persist entire Aggregates.

Poor.

```
PlaybackPositionRepository
PlaybackProgressRepository
```

Preferred.

```
PlaybackRepository
```

Repositories persist consistency boundaries.

Not individual implementation details.

Avoid Large Object Graphs

Large Aggregates often indicate poor modelling.

Poor.

```
Library
↓
Collections
↓
Media
↓
Metadata
↓
Playback
↓
Users
```

Better.

```
Library
↓
Collection IDs
Playback
↓
```

`Media ID`

Small Aggregates naturally improve scalability.

Aggregate Design Checklist

Before defining an Aggregate ask:

- What business rule am I protecting?
- What must always remain consistent?
- Can this Aggregate become smaller?
- Does another Aggregate own this concept?
- Am I modelling consistency or convenience?

If the answer is convenience, reconsider the boundary.

Mosaic Examples

Examples of Aggregates within Mosaic include:

`Library`

Responsible for:

- media ownership
- import state
- source configuration

`Playback Session`

Responsible for:

- playback progress
- completion state
- resume position

`Collection`

Responsible for:

- collection membership
- ordering
- user ownership

Each Aggregate protects one coherent set of business rules.

Anti-Patterns

The following practices are prohibited.

Giant Aggregates

```
Media Platform

↓

Everything
```

Cross-Aggregate Transactions

One transaction updating multiple Aggregates.

Shared Mutable Entities

Entities simultaneously belonging to multiple Aggregates.

Aggregate Bypass

Repositories modifying internal Entities directly.

Technical Aggregates

Grouping objects because they share a database table rather than a business invariant.

Mosaic Guidelines

Within Mosaic:

- Every Aggregate MUST protect one consistency boundary.
- Every Aggregate MUST have one Aggregate Root.
- Aggregate behaviour MUST enforce business invariants.
- Aggregates SHOULD remain small.
- Aggregates MUST communicate through identities or events.
- Cross-Aggregate consistency SHOULD be eventual.
- Repositories MUST persist Aggregates rather than individual Entities.
- Aggregate boundaries SHOULD follow business rules rather than object relationships.

Relationship to MEG

Entities answer:

> ***Who is this?***

Value Objects answer:

> ***What is this?***

Aggregates answer:

> ***What must always remain consistent together?***

The next chapter introduces the **Aggregate Root**, the only object permitted to expose and protect that consistency boundary.

Summary

Aggregates are the mechanism through which Domain-Driven Design preserves business correctness without sacrificing scalability.

Within Mosaic they provide:

- explicit consistency boundaries
- protected business invariants
- clear ownership
- small transactional units
- natural integration with the Event-Driven Runtime

Every Aggregate should exist because it protects an important business rule.

If it does not, it probably should not exist.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`07-value-objects.md`

Next File

`09-aggregate-roots.md`

Aggregate Roots

> *Every Aggregate has one guardian. Every business rule enters through it.*

Purpose

An Aggregate defines a business consistency boundary.

However, something must protect that boundary.

Without a single entry point:

- business rules become duplicated
- invariants become inconsistent
- internal entities become exposed
- external code bypasses business behaviour

Domain-Driven Design solves this by introducing the **Aggregate Root**.

Every Aggregate has exactly one Aggregate Root.

Every modification to the Aggregate passes through it.

This document defines how Aggregate Roots should be designed throughout the Mosaic platform.

Philosophy

Within Mosaic:

> ***The Aggregate Root protects the consistency of the Aggregate.***

The Aggregate Root is not merely the "first entity."

It is the only object responsible for maintaining the business integrity of the Aggregate.

Nothing outside the Aggregate should ever modify internal state directly.

What Is An Aggregate Root?

The Aggregate Root is the public entry point into an Aggregate.

Example.

```
Playback Aggregate
↓
PlaybackSession
↓
WatchProgress
↓
ResumePosition
```

Only:

```
PlaybackSession
```

is visible outside the Aggregate.

Everything else remains internal.

The Aggregate Root therefore acts as both:

- public API
- consistency guardian

One Root Per Aggregate

Every Aggregate MUST contain exactly one Aggregate Root.

Poor.

```
Library

↓

Media

↓

Collection

↓

Both Public
```

Better.

```
Library

↓

Library Aggregate Root

↓

Internal Entities
```

Multiple Aggregate Roots create ambiguous ownership.

Ownership should always be singular.

Martin Fowler summarises this rule succinctly: external references should point only to the Aggregate Root, which is responsible for maintaining the integrity of the Aggregate. [oai_citation:0‡martinfowler.com](https://martinfowler.com/bliki/DDD_Aggregate.html?utm_source=chatgpt.com)

Entry Point

Every modification to an Aggregate MUST occur through its Aggregate Root.

Poor.

```
Playback

↓
```

```
WatchProgress
↓
Update Directly
```

Preferred.

```
PlaybackSession
↓
Advance()
↓
WatchProgress Updated
```

Internal objects should never become publicly mutable.

Aggregate Boundary

The Aggregate Root defines the boundary between:

```
External World
```

and

```
Internal Business Rules
```

Everything outside the Aggregate sees only the Root.

Everything inside the Aggregate remains an implementation detail.

Business Authority

The Aggregate Root owns business authority.

Example.

```
Collection
↓
Add Media
```

The Collection decides:

- duplicates
- ordering
- ownership
- limits

External code simply requests the operation.

The Aggregate Root decides whether it is valid.

Protecting Invariants

Aggregate Roots exist primarily to protect invariants.

Example.

```
Playback Session

↓

Progress = 95%

↓

Advance()

↓

Completed = false
```

Later.

```
Progress = 100%

↓

Advance()

↓

Completed = true
```

The Aggregate Root guarantees these business rules always remain true.

Callers should never need to enforce them manually.

Internal Entities

Entities inside an Aggregate are implementation details.

Example.

```
PlaybackSession

■■■ ResumePosition

■■■ WatchProgress

■■■ PlaybackStatistics
```

External components should never hold references to these internal objects.

They interact only through:

```
PlaybackSession
```

References

Other Aggregates SHOULD reference only the Aggregate Root.

Example.

```
Collection
```

↓

```
MediaID
```

Not.

```
Collection
```

↓

```
MediaArtwork
```

↓

```
MediaMetadata
```

↓

```
PlaybackState
```

Cross-Aggregate references should always use identity.

Never internal object references.

This rule reduces coupling and preserves Aggregate boundaries. [oai_citation:1‡martinfowler.com](https://martinfowler.com/bliki/DDD_Aggregate.html?utm_source=chatgpt.com)

Aggregate API

Aggregate Roots should expose business behaviour.

Good.

```
[go]
collection.AddMedia(mediaID)

[go]
playback.Resume(position)
```

Poor.

```
[go]
collection.Items = append(...)
```

Public setters bypass business rules.

Business behaviour should always remain explicit.

Rich Behaviour

Aggregate Roots should become richer over time.

Initially.

```
Playback
```

↓

```
Resume()
```

Later.

```
Playback
```

↓

```
Resume()

Pause()

Seek()

Complete()

Restart()
```

As business understanding grows, behaviour should naturally accumulate.

Data alone rarely represents the business.

Persistence

Repositories persist Aggregate Roots.

Example.

```
PlaybackRepository

↓

Save(PlaybackSession)
```

Not.

```
PlaybackProgressRepository
```

Repositories should never persist internal Entities independently.

The Aggregate Root represents the transactional boundary.

Transactions

Every transaction should begin with the Aggregate Root.

```
Repository

↓

Load Aggregate Root

↓

Business Behaviour

↓

Save Aggregate Root
```

Internal Entities participate naturally.

The caller never coordinates them individually.

Domain Events

Aggregate Roots are the canonical source of Domain Events.

Example.

```
PlaybackSession

↓

Complete()

↓

PlaybackCompleted
```

The Domain Event is emitted because the Aggregate Root knows:

- the business transition occurred
- invariants remain satisfied
- state has become consistent

Infrastructure should not invent business events.

Validation

Aggregate Roots should validate every externally visible operation.

Example.

```
[go]
collection.AddMedia(mediaID)
```

Internally.

```
Already Exists?

↓

Maximum Size?

↓

User Owns Collection?

↓

Add
```

Validation belongs inside the Aggregate.

Callers should not duplicate business rules.

Aggregate Root Size

Aggregate Roots should remain cohesive.

Poor.

```
Playback

↓

Authentication

↓

Metadata
```

↓

```
Recommendations
```

Good.

```
Playback
```

↓

```
Playback Behaviour
```

The Aggregate Root should own one business concept.

Nothing more.

Constructors

Aggregate Roots SHOULD be created through constructors or factories.

Poor.

```
[go]
PlaybackSession{}
```

Preferred.

```
[go]
NewPlaybackSession(...)
```

Construction should establish a valid Aggregate immediately.

Invalid Aggregates should never exist.

Identity

The identity of the Aggregate is the identity of its Root.

Example.

```
PlaybackSessionID
```

Internal Entities may possess local identities if required.

Those identities should never escape the Aggregate boundary.

The Aggregate Root alone has global identity. [oai_citation:2‡Baeldung on Kotlin](https://www.baeldung.com/cs/aggregate-root-ddd?utm_source=chatgpt.com)

Aggregate Root Checklist

Before introducing an Aggregate Root ask:

- Does it own business consistency?
- Does every modification pass through it?
- Are internal Entities hidden?
- Does it expose business behaviour?

- Does it enforce invariants?
- Does it own Domain Events?

If any answer is "no", reconsider the Aggregate boundary.

Mosaic Examples

Examples of Aggregate Roots include:

`Library`

Responsible for:

- importing media
- source ownership
- library consistency

`PlaybackSession`

Responsible for:

- playback lifecycle
- progress
- completion
- resume position

`Collection`

Responsible for:

- membership
- ordering
- ownership
- duplicate prevention

Each Aggregate Root owns one coherent business consistency boundary.

Anti-Patterns

The following practices are prohibited.

Public Internal Entities

Allowing callers to modify child Entities directly.

Public Setters

Exposing Aggregate state without business validation.

Repository Per Internal Entity

Persisting child Entities independently.

Cross-Aggregate Object References

Holding references to another Aggregate's internal Entities.

Aggregate Root As Data Container

Aggregate Roots should own behaviour.

Not merely fields.

Business Rules Outside The Aggregate

Controllers, services or repositories enforcing Aggregate invariants.

The Aggregate Root owns those rules.

Mosaic Guidelines

Within Mosaic:

- Every Aggregate MUST have exactly one Aggregate Root.
- All modifications MUST pass through the Aggregate Root.
- Aggregate Roots MUST protect business invariants.
- Aggregate Roots MUST expose business behaviour.
- Internal Entities MUST remain hidden.
- Repositories MUST persist Aggregate Roots.
- Other Aggregates MUST reference Aggregate Roots by identity only.
- Aggregate Roots SHOULD publish Domain Events representing completed business transitions.

Relationship to MEG

Aggregates define:

> ***What must remain consistent together?***

Aggregate Roots define:

> ***Who is responsible for protecting that consistency?***

The next chapter introduces **Domain Services**, which model important business behaviour that belongs to the domain but naturally exists outside any single Aggregate.

Summary

Aggregate Roots are the guardians of business consistency.

They protect:

- invariants
- identity
- transactional boundaries
- business behaviour

Within Mosaic, every Aggregate Root exists for one reason:

> ***To ensure the business can never place the Aggregate into an invalid state.***

Everything else inside the Aggregate exists to support that responsibility.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`08-aggregates.md`

Next File

`10-domain-services.md`

Domain Services

> *Not every piece of business behaviour belongs to an Entity. When important business logic spans multiple concepts but still belongs to the domain, it belongs in a Domain Service.*

Purpose

Most business behaviour should naturally belong to:

- Entities
- Value Objects
- Aggregates

Occasionally, however, an important business operation does not belong naturally to any single domain object.

Examples include:

- matching recommendations
- calculating compatibility
- resolving media identity
- selecting metadata providers
- merging duplicate media

These behaviours are still part of the business domain.

They simply do not belong to one Aggregate.

Domain Services provide a home for this behaviour.

Philosophy

Within Mosaic:

> ***A Domain Service exists only when important business behaviour belongs to the domain but belongs to no individual Aggregate.***

A Domain Service should never become:

- a utility class
- a service layer
- an orchestration layer
- a collection of helper methods

It exists only to model business behaviour.

Eric Evans famously describes a Domain Service as appropriate when "it just isn't a thing." In other words, the behaviour is clearly part of the domain but does not naturally belong to an Entity or Value Object. [oai_citation:0‡O'Reilly Media](https://www.oreilly.com/library/view/implementing-domain-driven-design/97801

33039900/ch07.html?utm_source=chatgpt.com)

What Is A Domain Service?

A Domain Service models a business operation that:

- belongs to the domain
- involves multiple Aggregates
- has no natural owner
- represents important business knowledge

It is defined by behaviour.

Not by state.

Why Domain Services Exist

Consider:

```
Recommendation Engine
```

↓

```
Playback History
```

↓

```
Library
```

↓

```
Metadata
```

↓

```
User Preferences
```

Which Aggregate owns recommendation generation?

Not:

```
Playback
```

Not:

```
Library
```

Not:

```
Metadata
```

The behaviour belongs to the business.

Not to an individual Aggregate.

This is an ideal Domain Service.

Domain Service Characteristics

A Domain Service should:

- represent business behaviour
- remain stateless
- speak the ubiquitous language
- depend upon domain concepts
- return domain concepts

A Domain Service should **not**:

- persist data
- coordinate infrastructure
- expose HTTP
- publish runtime events directly
- manage transactions

Those concerns belong elsewhere.

Stateless By Design

Domain Services SHOULD be stateless.

Poor.

```
RecommendationService
↓
CurrentUser
↓
CurrentSession
↓
Cache
```

Better.

```
RecommendationService
↓
Input
↓
Business Calculation
↓
Result
```

State belongs to Aggregates.

Behaviour belongs to Domain Services.

Statelessness is one of the defining characteristics of a Domain Service. [oai_citation:1‡Domain-Driven Design Guide](https://ddd-practitioners.com/home/glossary/domain-service/?utm_source=chatgpt.com)

Domain Services Are Not Application Services

One of the most common misunderstandings in DDD is confusing Domain Services with Application Services.

Application Service.

```
Load Aggregate
```

↓

```
Call Domain
```

↓

```
Save Aggregate
```

↓

```
Publish Events
```

Domain Service.

```
Business Behaviour
```

Application Services coordinate.

Domain Services decide.

This distinction is fundamental.

Domain Services Are Not Utility Classes

Poor.

```
MediaUtils
```

```
LibraryHelper
```

```
RecommendationUtil
```

These names communicate implementation convenience.

Not business behaviour.

Instead:

```
RecommendationEngine
```

```
DuplicateResolver
```

```
MetadataMatcher
```

The name should describe business intent.

Behaviour Before Data

A Domain Service exists because of behaviour.

Not because of shared data.

Poor.

```
MetadataService

↓

Stores Metadata
```

That responsibility belongs to the Metadata Aggregate.

Better.

```
MetadataResolver

↓

Chooses Best Metadata Source
```

The behaviour belongs naturally to the domain.

Business Language

Domain Services should reinforce the ubiquitous language.

Good.

```
DuplicateResolver

RecommendationEngine

MetadataMatcher
```

Poor.

```
BusinessProcessor

DomainManager

Coordinator
```

Names should communicate business concepts.

Not technical roles.

Aggregate Collaboration

Domain Services frequently coordinate behaviour across multiple Aggregates.

Example.

```
Playback

↓

Library

↓
```

`RecommendationEngine`

↓

`Recommendations`

Notice:

The service does not own those Aggregates.

It merely applies business rules involving them.

Domain Services Should Be Rare

Most business behaviour belongs inside Aggregates.

Ask first:

> *Can this behaviour belong inside an Aggregate?*

If yes:

Do that.

Only introduce a Domain Service when no Aggregate can reasonably own the behaviour.

Large numbers of Domain Services often indicate an anemic domain model.

Domain Services May Depend Upon Repositories

Occasionally a Domain Service requires additional information.

Example.

`DuplicateResolver`

↓

`Repository`

↓

`Candidate Media`

This is acceptable when:

- the repository supports domain behaviour
- the behaviour genuinely spans multiple Aggregates

However:

Repositories should support the Domain Service.

Not become the Domain Service.

Domain Events

A Domain Service may cause Domain Events indirectly.

Example.

```
RecommendationEngine
```

↓

```
Recommendation Created
```

↓

```
Recommendation Aggregate
```

↓

```
RecommendationGenerated
```

The Domain Service performs business reasoning.

The Aggregate still owns business state and Domain Events.

Business facts should originate from Aggregates whenever practical.

Examples Within Mosaic

Appropriate Domain Services include:

```
MetadataMatcher
```

Determines the most appropriate metadata source.

```
DuplicateResolver
```

Determines whether imported media already exists.

```
RecommendationEngine
```

Calculates recommended media based on multiple business concepts.

```
CollectionOrderingPolicy
```

Determines business ordering rules for collections.

These services represent business knowledge.

Not infrastructure.

What Is Not A Domain Service?

The following are **not** Domain Services.

```
HTTPService
```

```
EmailSender
```

```
DatabaseManager
```

```
EventPublisher
```

These belong to infrastructure.

Likewise.

```
UserApplicationService
```

This coordinates a use case.

It does not model business behaviour.

Avoid God Services

Poor.

```
MediaDomainService
```

↓

```
Everything
```

Instead.

```
MetadataMatcher
```

```
DuplicateResolver
```

```
RecommendationEngine
```

Each service should represent one business capability.

Testing

Domain Services should be among the easiest components to test.

Because they are:

- stateless
- deterministic
- business focused

Tests should verify:

- business rules
- decision making
- edge cases

Infrastructure should remain outside the service.

Evolution

Domain Services often emerge naturally.

Initially.

```
Playback
```

↓

```
Recommendation Logic
```

Later.

```
Playback
```

↓

`RecommendationEngine`

As business understanding improves, behaviour that spans multiple Aggregates naturally becomes its own concept.

Extraction should follow understanding.

Not anticipation.

Anti-Patterns

The following practices are prohibited.

Generic Services

`BusinessService`

`DomainManager`

Stateful Services

Services storing mutable business state.

Infrastructure Services

Publishing events.

Sending emails.

Calling HTTP.

Writing SQL.

Anemic Aggregates

Moving Aggregate behaviour into Domain Services simply because "services exist."

Orchestration

Loading repositories.

Saving repositories.

Managing transactions.

These belong to Application Services.

Not Domain Services.

Mosaic Guidelines

Within Mosaic:

- Domain Services MUST model business behaviour.
- Domain Services SHOULD remain stateless.
- Domain Services SHOULD speak the ubiquitous language.
- Domain Services MUST NOT become application services.
- Domain Services MUST NOT own infrastructure concerns.
- Most business behaviour SHOULD remain inside Aggregates.
- Domain Services SHOULD remain rare.
- Domain Service names MUST communicate business intent.

Relationship to MEG

Entities own identity.

Value Objects own value.

Aggregates own consistency.

Aggregate Roots protect consistency.

Domain Services own business behaviour that naturally spans multiple Aggregates.

The next chapter introduces **Domain Events**, the mechanism through which important business facts leave the domain and become visible to the wider platform.

Summary

Domain Services exist for one reason:

> ***To model important business behaviour that belongs to the domain but belongs to no individual Aggregate.***

Used sparingly, they strengthen the domain model.

Used excessively, they usually indicate the domain model itself requires improvement.

Within Mosaic, Domain Services should therefore remain focused, expressive and unmistakably business-oriented.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`09-aggregate-roots.md`

Next File

`11-domain-events.md`

Domain Events

> *A Domain Event records that something important happened within the business. It is created by the domain before it becomes a runtime event.*

Purpose

Business behaviour changes business state.

Whenever an important business state transition occurs, the domain should record that fact.

These facts are known as **Domain Events**.

Domain Events represent significant moments within the business itself.

They exist independently of:

- event buses
- messaging systems
- transport protocols
- workers
- subscribers

This document defines how Domain Events are modelled within the Mosaic Domain Model and how they relate to the Reactive Runtime defined in MEG-002.

Philosophy

Within Mosaic:

> ***Business events originate in the domain. Runtime events transport them.***

This distinction is critical.

The domain determines:

> ***What happened.***

The runtime determines:

> ***How everyone else learns about it.***

The business owns facts.

The runtime owns communication.

What Is A Domain Event?

A Domain Event represents an important business fact.

Examples include:

```
PlaybackCompleted
```

```
MediaImported

CollectionCreated

MetadataCorrected
```

Each represents a completed business transition.

Each becomes part of the domain's history.

Domain Events Are Business Concepts

Domain Events belong entirely to the domain.

They should make sense even if:

- Go did not exist
- the runtime did not exist
- the application was rewritten tomorrow

Example.

```
PlaybackCompleted
```

is meaningful.

```
KafkaMessagePublished
```

is not.

Technology should never leak into the domain.

Business Before Runtime

The lifecycle is intentionally separated.

```
Business Behaviour

↓

Aggregate State Changes

↓

Domain Event

↓

Runtime Event

↓

Subscribers
```

Notice:

The runtime appears **after** the domain.

The domain remains completely unaware of transport.

Where Domain Events Come From

Domain Events originate from Aggregates.

Example.

```
PlaybackSession
```

↓

```
Complete()
```

↓

```
PlaybackCompleted
```

The Aggregate owns:

- business rules
- state transition
- business fact

Infrastructure merely transports the resulting event.

This follows the classic DDD pattern where Aggregates raise Domain Events as part of enforcing business behaviour. ([martinfowler.com](https://martinfowler.com/eaaDev/DomainEvent.html))

Domain Events Follow Behaviour

Domain Events should follow completed behaviour.

Correct.

```
Collection
```

↓

```
Add Media
```

↓

```
MediaAddedToCollection
```

Incorrect.

```
AddMediaRequested
```

The domain records facts.

Not requests.

One Business Transition

Every Domain Event should represent exactly one business transition.

Good.

```
PlaybackPaused
```

Poor.

```
PlaybackPausedAndRecommendationUpdated
```

Recommendations belong to another context.

One business fact.

One Domain Event.

Domain Events Are Immutable

Once raised, a Domain Event MUST NOT change.

Example.

```
PlaybackCompleted
```

↓

```
Immutable
```

If business understanding changes later:

```
PlaybackCorrected
```

becomes a new Domain Event.

History is additive.

Never rewritten.

Domain Events Belong To The Aggregate

Only the Aggregate owning the business state should raise the Domain Event.

Example.

```
Library Aggregate
```

↓

```
MediaImported
```

Metadata should never raise:

```
MediaImported
```

Ownership follows business ownership.

Always.

Domain Events Are Internal

One of the most important distinctions within Mosaic is:

```
Domain Event
```

≠

```
Runtime Event
```

The Domain Event exists inside the domain.

The Runtime Event exists outside it.

They frequently represent the same business fact.

They serve different architectural purposes.

Runtime Transformation

Conceptually.

```
PlaybackCompleted

↓

Domain Event

↓

Runtime Adapter

↓

Runtime Event

↓

Event Bus
```

The runtime adapter transforms domain concepts into transport concepts.

The Aggregate remains completely unaware of the Event Bus.

This separation keeps the domain pure while allowing the runtime to evolve independently.

Domain Events Should Be Small

Domain Events should contain only information describing the business fact.

Example.

```
PlaybackCompleted

↓

Playback ID

Media ID

User ID

Completed At
```

Avoid:

- retry counts
- trace identifiers
- routing information
- worker identifiers

Those belong to runtime infrastructure.

Domain Events Trigger Nothing

A Domain Event should never know:

- who receives it
- whether anyone receives it
- what happens next

It simply records:

> ***This happened.***

Everything afterwards belongs to the runtime.

Domain Events Are Not Integration Events

The domain should not model:

`WebhookSent`

`KafkaPublished`

`APIMessageSent`

These describe infrastructure.

Not business.

Instead.

`PlaybackCompleted`

The runtime determines how that fact is communicated externally.

Event Ordering

Domain Events follow business chronology.

Example.

`PlaybackStarted`

↓

`PlaybackPaused`

↓

`PlaybackResumed`

↓

`PlaybackCompleted`

The domain defines this sequence.

The runtime may deliver them differently.

Business chronology and delivery chronology remain separate concepts.

Event Collection

Aggregates SHOULD collect Domain Events during business execution.

Conceptually.

```
PlaybackSession

↓

Complete()

↓

Raise Domain Event

↓

Commit

↓

Runtime Publishes
```

Events should not leave the Aggregate until business consistency has been established.

This naturally complements the transactional boundaries defined earlier.

Domain Events And Transactions

Domain Events should only be published externally after successful persistence.

Example.

```
Aggregate Updated

↓

Commit Successful

↓

Publish Runtime Event
```

If persistence fails:

The Domain Event never leaves the Aggregate.

Business facts should never describe work that never became true.

Testing

Domain Events make business behaviour easy to test.

Example.

```
Playback Complete

↓

PlaybackCompleted Raised
```

Tests should verify:

- correct event
- correct payload
- correct timing

Testing business events is often simpler than testing infrastructure side effects.

Evolution

Domain Events evolve with business understanding.

Initially.

```
PlaybackCompleted
```

Later.

```
PlaybackCompleted

↓

CompletionSource

↓

CompletionReason
```

The business language evolves.

The event evolves with it.

The event should never become more technical.

Mosaic Examples

Examples of Domain Events include:

```
MediaImported

PlaybackStarted

PlaybackPaused

PlaybackCompleted

CollectionCreated

CollectionRenamed

MetadataCorrected

RecommendationGenerated
```

Every one represents an important business fact.

Anti-Patterns

The following practices are prohibited.

Infrastructure Events

`KafkaPublished`

`WebhookDelivered`

Commands

`RefreshMetadata`

`GenerateArtwork`

Mutable Events

Changing an event after it has been raised.

Runtime Dependencies

Aggregates importing:

- Event Bus
- Kafka
- NATS
- RabbitMQ

The domain should remain transport agnostic.

Publishing Before Commit

Publishing events before business state becomes durable.

Business Logic Inside Subscribers

Business behaviour should occur before the Domain Event exists.

Subscribers react.

They do not redefine history.

Mosaic Guidelines

Within Mosaic:

- Domain Events MUST describe completed business facts.
- Domain Events MUST originate from Aggregates.
- Domain Events MUST be immutable.
- Domain Events MUST remain independent of runtime infrastructure.
- Domain Events MUST NOT describe transport behaviour.

- Runtime Events SHOULD be derived from Domain Events.
- Domain Events SHOULD remain small and business focused.
- Domain Events SHOULD only leave the domain after successful persistence.

Relationship to MEG

MEG-002 introduced Runtime Events.

This chapter deliberately introduces an additional layer.

```
Aggregate

↓

Domain Event

↓

Runtime Translation

↓

Runtime Event

↓

Reactive Runtime
```

This separation is one of the most important architectural decisions within Mosaic.

It ensures:

- the business remains independent
- the runtime remains replaceable
- transport remains invisible to the domain

The domain owns meaning.

The runtime owns delivery.

Summary

Domain Events represent the moments that matter to the business.

They are not messages.

They are not notifications.

They are not transport.

They are simply immutable records that:

> ***Something important became true.***

Everything else, including retries, workers, event buses and subscribers, exists solely to communicate those business facts throughout the platform.

That distinction keeps the Mosaic Domain Model remarkably clean while allowing the Reactive Runtime to remain highly sophisticated.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`10-domain-services.md`

Next File

`12-repositories.md`

Repositories

> *A Repository gives the illusion that Aggregates simply exist. It hides persistence so the domain can remain focused on the business.*

Purpose

Business logic should never concern itself with:

- SQL
- PostgreSQL
- DuckDB
- Blob Storage
- Redis
- Filesystems

The domain should ask only one question:

> ***"Can I obtain this Aggregate?"***

Repositories answer that question.

They provide a collection-like abstraction over persistent storage while insulating the Domain Model from infrastructure concerns.

Philosophy

Within Mosaic:

> ***Repositories exist to persist Aggregates, not data.***

This distinction is fundamental.

Repositories do not manage:

- rows
- tables
- documents
- files

They manage:

- Aggregate Roots

Everything else is an implementation detail.

This reflects the classic DDD Repository pattern, whose purpose is to isolate the domain model from persistence concerns while presenting aggregates as though they were held in an in-memory collection. [oai_citation:0‡O'Reilly Media](https://www.oreilly.com/library/view/implementing-domain-driven-design/9780133039900/ch12.html?utm_source=chatgpt.com)

What Is A Repository?

A Repository provides the illusion that Aggregates are stored within an in-memory collection.

Conceptually.

```
Playback Repository

↓

Load Playback Session

↓

Modify Aggregate

↓

Save Aggregate
```

How that Aggregate is stored is irrelevant to the domain.

Why Repositories Exist

Without Repositories:

```
Playback

↓

SQL

↓

Database

↓

Rows

↓

Business Logic
```

The business now understands infrastructure.

Instead:

```
Playback

↓

Repository

↓

Persistence
```

The Aggregate remains infrastructure agnostic.

Repository Responsibilities

Repositories are responsible for:

- loading Aggregates
- saving Aggregates
- removing Aggregates
- querying Aggregate identity

Repositories are **not** responsible for:

- business rules
- validation
- orchestration
- transactions
- event publication

Business belongs to the domain.

Persistence belongs to infrastructure.

One Repository Per Aggregate

Every Aggregate Root SHOULD have one Repository.

Example.

```
Playback Aggregate

↓

PlaybackRepository

Library Aggregate

↓

LibraryRepository
```

Not:

```
PlaybackPositionRepository

WatchProgressRepository
```

Repositories persist Aggregate boundaries.

Not implementation details.

This one-repository-per-aggregate-root approach is one of the core recommendations of DDD because repositories preserve aggregate consistency boundaries. [oai_citation:1‡Microsoft Learn](https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design?utm_source=chatgpt.com)

Aggregate Roots Only

Repositories should load and save Aggregate Roots.

Poor.

```
PlaybackProgress

↓

Repository
```

Preferred.

```
PlaybackSession

↓

Repository
```

Internal Entities remain internal.

The Aggregate Root protects consistency.

Collection Semantics

Repositories should feel like collections.

Examples.

```
Find()

Save()

Delete()

Exists()
```

They should not expose database operations.

Poor.

```
ExecuteSQL()

RunQuery()

UpdateRow()
```

These reveal persistence.

Not business behaviour.

Domain Language

Repository APIs should speak the ubiquitous language.

Good.

```
[go]
libraryRepository.FindByID(...)

[go]
playbackRepository.Save(...)
```

Poor.

```
[go]
database.Select(...)

[go]
repository.Execute(...)
```

Names should communicate business intent.

Repository Interfaces

Repository interfaces belong to the domain.

Implementations belong to infrastructure.

Example.

```
Domain

↓

PlaybackRepository Interface

Infrastructure

↓

PostgresPlaybackRepository
```

The Domain knows nothing about PostgreSQL.

Infrastructure knows everything.

This separation preserves persistence ignorance while allowing different infrastructure implementations. [oai_citation:2‡Stack and System](https://stackandsystem.com/series/domain-driven-design/9-repositories-in-domain-driven-design?utm_source=chatgpt.com)

Persistence Ignorance

Aggregates should remain completely unaware of persistence.

Poor.

```
[go]
playback.Save()
```

Preferred.

```
[go]
repository.Save(playback)
```

Persistence should always occur outside the Aggregate.

The domain models business.

Infrastructure models storage.

Transactions

Repositories participate in transactions.

They do not own them.

Typical flow.

```
Load Aggregate

↓

Business Behaviour

↓

Save Aggregate
```

The Repository persists the Aggregate.

The Aggregate determines what changed.

Queries

Repositories exist primarily for write models.

Complex reporting queries often belong elsewhere.

Example.

```
PlaybackRepository

↓

Playback Aggregate
```

versus.

```
Continue Watching View

↓

Read Model
```

Repositories should not become reporting engines.

This aligns naturally with CQRS where rich queries are separated from aggregate persistence.

Avoid Generic Repositories

Poor.

```
[go]
type Repository[T any]
```

While technically elegant, generic repositories frequently describe persistence rather than business behaviour.

Instead.

```
PlaybackRepository

LibraryRepository
```

```
CollectionRepository
```

Business language should always dominate technical abstraction.

Repository Methods

Repositories SHOULD expose intention-revealing operations.

Examples.

```
[go]
FindByID()

[go]
FindBySource()

[go]
Exists()
```

Avoid exposing generic database operations.

Repositories should describe business retrieval.

Not storage mechanics.

Repository Scope

Repositories should remain small.

Poor.

```
PlaybackRepository

↓

Save

Find

Export

Metrics

Analytics

Search

Cache

Notifications
```

Better.

```
PlaybackRepository

↓

Load

Save

Delete
```

Business behaviour belongs elsewhere.

Repositories Do Not Orchestrate

Poor.

```
Repository

↓

Load

↓

Modify Aggregate

↓

Publish Events

↓

Call API
```

Repositories should simply persist Aggregates.

Everything else belongs to:

- Aggregates
- Domain Services
- Application Services
- Runtime

Multiple Storage Engines

One of Mosaic's defining architectural characteristics is multiple storage technologies.

Examples include:

- PostgreSQL
- DuckDB
- Blob Storage
- Filesystem
- MOS Files

The Domain should remain unaware of all of them.

Every implementation should satisfy the same Repository contract.

Changing persistence should never require changing business behaviour.

Repository Lifetime

Repositories should remain lightweight.

They should not:

- cache business state
- maintain sessions
- own transactions
- retain Aggregate instances

Every call should remain explicit.

Repositories are gateways.

Not application state.

Testing

Repositories should be replaceable during testing.

Example.

`PlaybackRepository`

↓

`In-Memory Repository`

Business tests should not require:

- PostgreSQL
- DuckDB
- Blob Storage

The domain should remain testable independently of infrastructure.

Anti-Patterns

The following practices are prohibited.

Generic Repository

`Repository<T>`

used for every Aggregate.

Repository Per Entity

Repositories for child Entities.

SQL In The Domain

`SELECT ...`

appearing within business logic.

Business Logic In Repositories

Repositories deciding business behaviour.

Returning Persistence Models

Repositories returning database rows rather than Aggregates.

Cross-Aggregate Repositories

One Repository managing multiple unrelated Aggregate Roots.

Mosaic Guidelines

Within Mosaic:

- Every Aggregate Root SHOULD have one Repository.
- Repositories MUST persist Aggregate Roots.
- Repository interfaces SHOULD belong to the Domain.
- Repository implementations MUST belong to Infrastructure.
- Repositories MUST speak the ubiquitous language.
- Repositories MUST remain persistence focused.
- Business behaviour MUST remain outside Repositories.
- The Domain MUST remain unaware of storage technologies.

Relationship to MEG

Aggregates own consistency.

Aggregate Roots protect consistency.

Repositories preserve consistency.

The next chapter introduces **Factories**, which solve the complementary problem of constructing complex Aggregates in valid business states before they are ever persisted.

Summary

Repositories are one of the most misunderstood patterns in Domain-Driven Design.

They are not database wrappers.

They are not DAOs.

They are not query builders.

Within Mosaic, they exist for one purpose:

> ***Provide the domain with the illusion that Aggregates simply exist, while keeping every persistence concern outside the business model.***

When implemented correctly, changing PostgreSQL to another storage engine should require changes only within infrastructure.

The business should never notice.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`11-domain-events.md`

Next File

`13-factories.md`

Factories

> *Construction is not business behaviour. Complex creation should be hidden so the domain begins life in a valid state.*

Purpose

Some domain objects are simple to construct.

Others are not.

Creating an Aggregate may require:

- multiple Entities
- several Value Objects
- validation
- business rules
- default state
- invariant enforcement

Embedding this logic inside constructors or Application Services quickly leads to duplicated and inconsistent creation logic.

Factories solve this problem.

They encapsulate complex object creation while ensuring every new domain object begins life in a valid business state.

Philosophy

Within Mosaic:

> ***Factories create valid domain objects. They do not perform business behaviour.***

Factories exist to solve one problem:

> ***How should a complex business object be created correctly?***

They should not become:

- service layers
- repositories
- orchestrators
- builders
- workflow engines

Their responsibility begins and ends with construction.

Why Factories Exist

Consider creating a new Library.

Requirements might include:

- unique identifier
- default settings
- root collection
- default permissions
- import configuration

Without a Factory:

```
[go]
library := &Library{
    ...
}
```

The caller must know:

- every required field
- every default
- every invariant

This knowledge becomes duplicated throughout the application.

A Factory centralises that knowledge.

DDD factories exist specifically to separate object construction from object use while ensuring newly created Aggregates satisfy all invariants. [oai_citation:0‡O'Reilly Media](https://www.oreilly.com/library/view/implementing-domain-driven-design/9780133039900/ch11lev1sec1.html?utm_source=chatgpt.com)

What Is A Factory?

A Factory is responsible for creating:

- Entities
- Aggregates
- occasionally Value Objects

in a valid business state.

A Factory does **not** own the resulting object.

Ownership transfers immediately to the caller.

Construction Is Not Behaviour

Factories should create objects.

They should not perform business operations.

Good.

```
Create Library
```

Poor.

```
Create Library

↓

Import Media

↓

Generate Artwork

↓

Publish Events
```

Creation and behaviour should remain separate.

Valid From Birth

Factories SHOULD guarantee that every created object satisfies its business invariants.

Poor.

```
[go]
library := &Library{}

library.Name = name

library.Owner = owner

library.Validate()
```

Preferred.

```
[go]
library, err := NewLibrary(...)
```

Invalid objects should never exist, even temporarily.

Aggregate Creation

Factories are most valuable when creating Aggregates.

Example.

```
Library Factory

↓

Library

↓

Default Collection

↓

Configuration

↓
```

```
Policies
```

The caller receives a complete Aggregate.

Not a partially initialised object.

Factory Or Constructor?

The simplest solution should always be preferred.

Use a constructor when:

- creation is straightforward
- invariants are simple
- dependencies are minimal

Example.

```
[go]
NewDuration(...)
```

Use a Factory when:

- multiple objects must be assembled
- creation logic is complex
- business rules determine construction
- Aggregate assembly becomes non-trivial

Factories should emerge naturally.

Not automatically.

Factory Methods

Many Aggregates naturally create their own internal objects.

Example.

```
Collection
```

↓

```
AddMedia()
```

↓

```
CollectionItem Created
```

The Aggregate Root itself acts as the Factory.

This is often preferable to introducing another type.

Only introduce a dedicated Factory when creation logic genuinely becomes complex.

Evans and Vernon both encourage using Aggregate methods as factories where the creation naturally belongs to the Aggregate itself. [oai_citation:1‡O'Reilly Media](https://www.oreilly.com/library/view/implementing-domain-driven-design/9780133039900/ch11.html?utm_source=chatgpt.com)

Domain Language

Factory names should communicate business intent.

Good.

```
LibraryFactory
```

```
PlaybackFactory
```

```
CollectionFactory
```

Poor.

```
ObjectFactory
```

```
EntityBuilder
```

```
GenericFactory
```

Names should reinforce the ubiquitous language.

Factory Responsibilities

Factories MAY:

- validate creation rules
- assemble child Entities
- construct Value Objects
- assign identities
- establish defaults

Factories MUST NOT:

- persist objects
- publish events
- start workflows
- access HTTP
- perform infrastructure operations

Their responsibility ends once construction completes.

Factory Dependencies

Factories should depend only upon:

- domain concepts
- domain policies
- domain services

They should never depend directly upon:

- databases
- message buses
- HTTP
- file systems

Construction remains a domain concern.

Persistence does not.

Identities

Factories frequently create identities.

Example.

```
LibraryFactory

↓

Generate LibraryID

↓

Create Aggregate
```

The mechanism used to generate the identifier is an implementation concern.

The resulting identity is a domain concept.

Value Objects

Simple Value Objects rarely require Factories.

Usually:

```
[go]
duration := NewDuration(...)
```

is entirely sufficient.

Dedicated Factories for simple immutable Value Objects generally introduce unnecessary complexity.

Aggregate Integrity

Factories should create complete Aggregates.

Poor.

```
Library

↓

Missing Root Collection
```

Preferred.

```
Library
↓
Root Collection
↓
Configuration
↓
Ready
```

The caller should never be responsible for "finishing" Aggregate construction.

Application Services

Application Services frequently use Factories.

Example.

```
Command
↓
Application Service
↓
Library Factory
↓
Repository
↓
Runtime
```

Notice:

The Application Service coordinates.

The Factory constructs.

The Repository persists.

Responsibilities remain distinct.

Testing

Factories should be easy to test.

Typical tests verify:

- valid construction
- invariant enforcement
- default values
- invalid inputs

Factories should remain deterministic.

The same inputs should produce equivalent business objects.

Evolution

Factories should evolve alongside the domain.

Initially.

```
NewLibrary(...)
```

Later.

```
LibraryFactory

↓

Policies

↓

Defaults

↓

Configuration
```

Complexity should move into the Factory only when it genuinely exists.

Do not anticipate complexity prematurely.

What Is Not A Factory?

The following are **not** Factories.

```
Repository
```

Repositories retrieve and persist.

```
Application Service
```

Coordinates use cases.

```
Domain Service
```

Models business behaviour.

```
Builder
```

Supports incremental object construction.

Factories produce complete domain objects.

Mosaic Examples

Appropriate Factory responsibilities include:

```
LibraryFactory
```

↓

`Create new Library Aggregate`

`CollectionFactory`

↓

`Create default Collection hierarchy`

`PlaybackFactory`

↓

`Create new Playback Session`

`ImportFactory`

↓

`Create Import Job Aggregate`

Each creates a valid business object.

None execute business workflows.

Anti-Patterns

The following practices are prohibited.

Partially Constructed Aggregates

Returning objects that require additional setup before becoming valid.

Factory As Service

Factories performing business operations after construction.

Infrastructure Inside Factories

Factories executing SQL, HTTP or message publication.

Generic Factory

`ObjectFactory`

creating unrelated domain concepts.

Builders For Everything

Introducing Builder patterns where a simple constructor or Aggregate factory method is clearer.

Mosaic Guidelines

Within Mosaic:

- Factories SHOULD create valid Aggregates.
- Factories SHOULD encapsulate complex construction.
- Factories MUST enforce creation invariants.
- Factories MUST remain infrastructure independent.
- Aggregate factory methods SHOULD be preferred where creation naturally belongs to the Aggregate.
- Constructors SHOULD be preferred for simple creation.
- Factories MUST NOT perform persistence.
- Factories MUST NOT publish events.

Relationship to MEG

Repositories answer:

> ***How do I retrieve and persist Aggregates?***

Factories answer:

> ***How do I create Aggregates correctly?***

Together they define the beginning and end of an Aggregate's lifecycle.

The next chapter introduces **Domain Invariants**, the business rules that every Aggregate, Factory and Entity must protect throughout that lifecycle.

Summary

Factories exist to protect one of the most important moments in a domain object's life:

> ***Its creation.***

Within Mosaic, every Aggregate should begin life:

- complete
- valid
- consistent
- expressive

By encapsulating construction inside Factories, the domain remains focused on business concepts rather than construction mechanics, and every new object enters the system already satisfying the rules that define it.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`12-repositories.md`

Next File

`14-domain-invariants.md`

Domain Invariants

> *Business rules are not suggestions. They define what it means for the domain to be correct.*

Purpose

Every business domain contains rules that must always remain true.

Examples include:

- A playback position cannot exceed the media duration.
- A Collection cannot contain duplicate media.
- A User cannot own another user's Library.
- Watch progress cannot be negative.

These rules define the integrity of the business.

Domain-Driven Design refers to these rules as **Domain Invariants**.

This document defines how Domain Invariants should be identified, protected and enforced throughout the Mosaic platform.

Philosophy

Within Mosaic:

> ***Invalid business state should be impossible to represent.***

The responsibility of the Domain Model is not merely to process data.

Its responsibility is to prevent invalid business state from ever existing.

If an Aggregate can enter an invalid state, the model has failed.

What Is A Domain Invariant?

A Domain Invariant is a business rule that must always be true.

Examples include:

```
Playback Progress

≤

Media Duration

Collection

↓

Unique Media

User

↓
```

```
Unique Username
```

The business assumes these rules are always satisfied.

The software should enforce them accordingly.

Why Invariants Exist

Without invariants:

```
Playback

↓

Duration = 90 Minutes

↓

Progress = 120 Minutes
```

The system now contains impossible business state.

Every subsequent operation becomes more complicated because every consumer must now defend against invalid data.

Instead:

The invalid state should never be constructible.

Business Rules

Only genuine business rules become invariants.

Examples.

Good.

```
Library Name Required

Playback Cannot Complete Before It Starts
```

Poor.

```
Maximum HTTP Connections

Database Timeout
```

Infrastructure rules belong elsewhere.

Invariants belong to the business.

Invariants Belong To The Domain

Business rules should never be enforced solely by:

- controllers
- HTTP handlers
- repositories

- UI validation

Poor.

```
HTTP

↓

Validate

↓

Domain
```

Preferred.

```
Domain

↓

Validate

↓

Always Correct
```

Infrastructure may validate for convenience.

The Domain validates for correctness.

Aggregate Responsibility

Aggregates own invariants.

Example.

```
PlaybackSession

↓

Advance()

↓

Validate Progress

↓

Apply Change
```

The Aggregate protects itself.

External callers should never be responsible for enforcing business rules.

Entity Responsibility

Entities may also enforce local invariants.

Example.

```
Collection

↓

Rename()
```

↓

```
Name Cannot Be Empty
```

The Entity understands the rule because it owns the concept.

Business rules should live with the business object they describe.

Value Objects

Value Objects frequently enforce invariants during construction.

Example.

```
[go]
duration := NewDuration(seconds)
```

Possible rules:

- non-negative
- finite
- valid range

If construction fails:

The Value Object never exists.

This is preferable to creating invalid values and validating them later.

Construction

Factories should also enforce invariants.

Poor.

```
Create Aggregate
```

↓

```
Validate Later
```

Preferred.

```
Validate
```

↓

```
Create Aggregate
```

↓

```
Always Valid
```

Every Aggregate should begin life in a valid business state.

Factories and constructors should enforce this from the outset.

Protecting State

All state mutation should occur through business behaviour.

Poor.

```
[go]
playback.Progress = progress
```

Preferred.

```
[go]
playback.Advance(progress)
```

The Aggregate now controls:

- validation
- state transition
- invariant enforcement

Business correctness becomes automatic.

Preventing Invalid State

The preferred strategy is prevention.

Not correction.

Poor.

```
Invalid State
```

↓

```
Fix Later
```

Preferred.

```
Reject Invalid Change
```

If invalid state never exists, downstream logic becomes dramatically simpler.

Local Invariants

Some invariants exist entirely inside one Aggregate.

Example.

```
Collection
```

↓

```
Duplicate Media Prohibited
```

Only the Collection Aggregate needs to understand this rule.

No other capability participates.

These invariants should remain local.

Cross-Aggregate Rules

Some business rules appear to span multiple Aggregates.

Example.

```
Library Storage Quota

Subscription Limits
```

These should rarely be enforced through distributed transactions.

Instead.

```
Aggregate

↓

Domain Event

↓

Another Aggregate

↓

Eventually Consistent
```

Only rules requiring immediate consistency belong inside one Aggregate.

Everything else should generally become eventual consistency.

Expressing Invariants

Good business behaviour makes invariants obvious.

Example.

Instead of:

```
[go]
SetCompleted(true)
```

Prefer.

```
[go]
Complete()
```

The Aggregate now decides whether completion is valid.

Intent-revealing methods naturally reinforce business rules.

Validation Order

Business behaviour should generally follow this pattern.

```
Validate

↓

Modify State

↓
```

```
Raise Domain Event
```

If validation fails:

State does not change.

No Domain Event is raised.

Business history remains correct.

Domain Events

Domain Events should only be raised after invariants have been satisfied.

Poor.

```
Raise Event

↓

Validation Fails
```

Preferred.

```
Validation

↓

State Change

↓

Raise Event
```

Events describe completed truth.

Not attempted operations.

Persistence

Repositories should never repair broken Aggregates.

Poor.

```
Repository

↓

Fix Invalid Aggregate

↓

Save
```

If an Aggregate reaches persistence in an invalid state:

The bug already occurred.

Repositories persist correctness.

They do not create it.

Infrastructure Validation

UI validation.

HTTP validation.

JSON validation.

These improve user experience.

They do **not** replace Domain validation.

Every business rule should still exist inside the Domain.

Never trust external validation alone.

Testing Invariants

Every invariant SHOULD have dedicated tests.

Examples.

```
Progress Cannot Exceed Duration
```

```
Collection Rejects Duplicates
```

```
Empty Library Name Rejected
```

Tests should verify:

- valid behaviour
- invalid behaviour
- edge cases

Business correctness deserves explicit verification.

Evolving Rules

Business rules change.

For example.

Initially.

```
Collection
```

↓

```
Maximum 100 Items
```

Later.

```
Unlimited Collections
```

The invariant changes.

The Domain evolves.

Implementation follows.

The model should evolve with business understanding.

Common Mosaic Invariants

Examples include:

Playback.

- Progress ≥ 0
- Progress ≤ Duration
- Completed implies Progress = Duration

Library.

- Library has an owner.
- Root collection always exists.

Collection.

- Media references are unique.
- Name cannot be empty.

Metadata.

- Primary artwork always exists.
- Source provider recorded.

User.

- Identity immutable.
- Username unique.

These examples are illustrative rather than exhaustive.

Anti-Patterns

The following practices are prohibited.

Validation Only In Controllers

Business rules enforced exclusively by HTTP.

Public Mutable State

Allowing callers to bypass business behaviour.

Invalid Aggregates

Allowing construction before validation.

Repository Repair

Repositories silently fixing broken business objects.

Duplicate Validation

Implementing business rules independently in:

- UI
- HTTP
- Services
- Domain

The Domain remains authoritative.

Business Rules In Infrastructure

Embedding business validation inside persistence or transport layers.

Mosaic Guidelines

Within Mosaic:

- Every business invariant MUST be enforced by the Domain.
- Aggregates MUST protect Aggregate invariants.
- Value Objects SHOULD validate themselves.
- Factories MUST construct valid Aggregates.
- Invalid state MUST NOT be representable.
- Domain Events MUST only follow successful validation.
- Repositories MUST persist already-valid Aggregates.
- Infrastructure validation MUST complement, not replace, Domain validation.

Relationship to MEG

Factories ensure Aggregates begin life correctly.

Invariants ensure they remain correct.

Together they guarantee:

```
Construction
```

↓

```
Valid Aggregate
```

↓

```
Business Behaviour
↓
Valid Aggregate
↓
Persistence
```

The next chapter introduces **Modelling Guidelines**, bringing together the concepts introduced throughout MEG-003 into practical modelling advice for engineers designing new business capabilities.

Summary

Domain Invariants are the rules that define business correctness.

They are not optional.

They are not advisory.

They are the reason the Domain Model exists.

Within Mosaic, every Aggregate, Entity and Value Object should actively prevent invalid business state rather than attempting to repair it afterwards.

Correctness should be the natural outcome of the model itself.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`13-factories.md`

Next File

`15-modelling-guidelines.md`

Modelling Guidelines

> *The purpose of modelling is not to describe software. It is to understand the business.*

Purpose

The previous chapters introduced the building blocks of Domain-Driven Design:

- Ubiquitous Language
- Bounded Contexts
- Entities
- Value Objects
- Aggregates
- Aggregate Roots
- Domain Services
- Domain Events
- Repositories
- Factories
- Domain Invariants

This document brings those concepts together into practical modelling guidance.

Its purpose is to help engineers answer one question.

> ***"How should I model a new business capability?"***

Philosophy

Within Mosaic:

> ***Model the business as it exists today. Allow tomorrow's understanding to evolve naturally.***

Good models emerge through understanding.

They are rarely designed perfectly on the first attempt.

Model discovery is continuous.

Start With The Business

Every modelling exercise should begin with business questions.

Examples include:

- What problem is being solved?
- What language do users use?

- What concepts exist?
- What behaviours exist?
- What business rules always remain true?

Do not begin with:

- database tables
- HTTP endpoints
- events
- packages

Technology follows the model.

Never the reverse.

Find The Ubiquitous Language

Before writing code, identify the language.

Ask:

- What nouns exist?
- What verbs exist?
- Which concepts appear repeatedly?
- Which words are ambiguous?

The ubiquitous language becomes:

- documentation
- code
- package names
- event names

Every future engineering decision builds upon this vocabulary.

Identify The Bounded Context

Ask:

> ***Who owns this concept?***

Every new concept belongs to one Bounded Context.

Examples.

```
Playback

Metadata

Library
```

If ownership is unclear:

Do not continue modelling.

Clarify ownership first.

Identify The Aggregate

Ask:

> ***What business rules must always remain consistent together?***

Not:

> ***Which objects reference one another?***

Consistency determines Aggregate boundaries.

Object graphs do not.

This is one of the central heuristics for aggregate design in Domain-Driven Design.
([dddcommunity.org](https://dddcommunity.org/wp-content/uploads/files/pdf_articles/Vernon_2011_1.pdf))

Find The Aggregate Root

Every Aggregate should answer:

> ***Which object protects the business rules?***

The answer becomes the Aggregate Root.

Everything else remains internal.

If multiple objects appear equally important:

The Aggregate boundary probably requires refinement.

Identify Entities

Ask:

> ***Which concepts possess identity?***

Examples.

```
Media
```

```
Collection
```

```
Playback Session
```

Identity determines Entities.

Not storage.

Identify Value Objects

Ask:

> ***Which concepts are defined entirely by their value?***

Examples.

```
Duration

Language

Resolution
```

Whenever identity is unnecessary:

Prefer a Value Object.

Model Behaviour

Ask:

> ***What does this concept do?***

Not:

> ***What fields does it contain?***

Poor.

```
Media

↓

Fields
```

Better.

```
Media

↓

Rename()

Archive()

Move()

Restore()
```

Business behaviour should dominate the model.

Protect Invariants

Every Aggregate should answer:

> ***What business rules must never become false?***

Those rules become Domain Invariants.

They should be enforced:

- automatically
- consistently
- immediately

Business correctness should never depend upon callers remembering validation.

Raise Domain Events

Whenever an important business fact becomes true:

Raise a Domain Event.

Example.

```
Playback

↓

Complete()

↓

PlaybackCompleted
```

Do not ask:

> *Should other capabilities care?*

That question belongs to the runtime.

Introduce Domain Services Carefully

Ask:

> ***Does this behaviour naturally belong to an Aggregate?***

If yes:

Keep it there.

If no:

Consider a Domain Service.

Domain Services should remain rare.

They represent important business behaviour that has no natural owner.

Introduce Repositories Last

Repositories exist only after the Domain Model exists.

Do not begin with persistence.

Model first.

Persist later.

Repositories support the Domain.

They do not define it.

Model Small

Prefer:

```
Playback
```

over:

```
Media Platform
```

Prefer:

```
RecommendationEngine
```

over:

```
BusinessManager
```

Small models are:

- easier to understand
- easier to evolve
- easier to test

Large models usually hide multiple responsibilities.

Evolve Continuously

Do not expect the first model to remain correct.

As understanding improves:

- rename concepts
- split Aggregates
- refine language
- move behaviour
- redefine boundaries

Changing the model is evidence of improved understanding.

Not failure.

Resist Technical Thinking

Poor.

```
DTO
```

↓

```
Entity
```

```
↓
Controller
↓
Repository
```

Better.

```
Playback
↓
Complete()
↓
PlaybackCompleted
```

The second model communicates the business.

The first communicates implementation.

The Domain should remain free from technical vocabulary.

Avoid Premature Generalisation

Do not model hypothetical future concepts.

Poor.

```
Universal Media Item
↓
Supports Everything
↓
Maybe Useful Later
```

Instead.

```
Movie
Series
Book
```

Generalisation should emerge naturally.

Not speculatively.

Draw The Model

Before implementing:

Draw.

Example.

```
Playback Session
■■■ Progress
```

■■■ Duration

■■■ Resume Position

Simple diagrams frequently reveal:

- missing concepts
- incorrect ownership
- unnecessary coupling

Visual modelling is often cheaper than implementation.

Ask Better Questions

Good modelling questions include:

- What does the business call this?
- Who owns this concept?
- What happens when this changes?
- What business rules exist?
- What events naturally occur?
- What cannot be allowed to happen?

Good questions produce good models.

Modelling Checklist

Before implementing a new capability ask:

- [ ] Is the ubiquitous language clear?
- [ ] Does the concept belong to one Bounded Context?
- [ ] Is ownership obvious?
- [ ] Have Entities been distinguished from Value Objects?
- [ ] Are Aggregate boundaries driven by consistency?
- [ ] Does one Aggregate Root protect the Aggregate?
- [ ] Are Domain Events identified?
- [ ] Are invariants explicit?
- [ ] Does the model avoid infrastructure concerns?
- [ ] Can another engineer explain the model in business terms?

If any answer is "no", continue modelling.

Implementation should wait.

Common Modelling Mistakes

Avoid:

- modelling databases
- modelling APIs
- modelling transport
- modelling frameworks
- modelling packages

Instead model:

- behaviour
- business rules
- ownership
- identity
- language

The software should become an expression of the business.

Not the implementation.

Mosaic Guidelines

Within Mosaic:

- Model the business before the software.
- Prefer rich domain models.
- Behaviour SHOULD remain inside the domain.
- Aggregates SHOULD remain small.
- Ubiquitous Language SHOULD remain consistent.
- Infrastructure MUST remain outside the domain.
- Models SHOULD evolve continuously.
- Simplicity SHOULD always be preferred over speculative flexibility.

Relationship to MEG

This chapter completes the tactical modelling guidance of MEG-003.

The remaining documents describe:

- architectural reasoning (ADRs)
- contributor expectations

- terminology
- references

The next engineering specification, **MEG-004 – Hexagonal Architecture**, will describe how these Domain Models interact with infrastructure without compromising their integrity.

Together, MEG-003 and MEG-004 define both:

- **what** the business model is, and
- **how** it remains protected from technical concerns.

Summary

Domain-Driven Design is not a collection of patterns.

It is a way of thinking.

Within Mosaic, every model should answer one simple question:

> ***"Does this make the business easier to understand?"***

If the answer is yes, the model is probably improving.

If the answer is no, the implementation is almost certainly modelling technology rather than the business.

That distinction is the difference between software that merely works and software that continues to evolve gracefully for years.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`14-domain-invariants.md`

Next File

`16-adrs.md`

Architectural Decision Records (ADRs)

> *The Domain Model is the most valuable intellectual asset within the platform. Changes to that model should always be intentional, documented and historically traceable.*

Purpose

The Mosaic Domain Model defines:

- business language
- business ownership
- business behaviour
- business boundaries

Changes to these concepts affect every layer built upon them.

Architectural Decision Records (ADRs) ensure that significant domain modelling decisions are preserved alongside the architecture itself.

Future contributors should understand not only:

> ***What the model is***

but also:

> ***Why it became that model.***

Philosophy

Within Mosaic:

> ***The domain should evolve through deliberate understanding rather than accidental implementation.***

Changes to the Domain Model should represent improved understanding of the business.

Not convenience.

Not implementation pressure.

Not framework limitations.

Why Domain ADRs Matter

Business understanding evolves.

For example.

Initially.

```
Media
```

Later.

```
Movie
```

```
Series

Book

Music
```

Why did the model change?

Without documentation:

Nobody knows.

Domain ADRs preserve the reasoning behind those decisions.

When An ADR Is Required

A Domain ADR SHOULD be created whenever a decision changes:

- ubiquitous language
- bounded contexts
- aggregate boundaries
- entity ownership
- business terminology
- domain events
- invariants
- context relationships
- repository ownership

If the decision changes how the business is understood, it probably deserves an ADR.

Examples

Examples of Domain ADRs include:

```
ADR-001

Library Is The Core Aggregate

ADR-002

Playback As Independent Context

ADR-003

Metadata Ownership

ADR-004

Continue Watching Model

ADR-005

Recommendation Domain

ADR-006

Media Identity Strategy
```

```
ADR-007

Collection Ownership
```

These decisions shape the business itself.

They should remain discoverable.

Domain Stability

The Domain Model should evolve more slowly than implementation.

Changing:

```
Repository

↓

Implementation
```

is inexpensive.

Changing:

```
Library

↓

Business Meaning
```

affects:

- runtime
- APIs
- storage
- events
- documentation
- extensions

Domain evolution therefore deserves significantly greater care.

ADR Structure

Every Domain ADR SHOULD contain:

```
Title

↓

Status

↓

Context

↓

Business Problem
```

```
↓

Options

↓

Decision

↓

Business Consequences

↓

Migration

↓

Related Specifications
```

Notice:

The problem is framed in business terms.

Not implementation terms.

Context

The Context section should describe:

- existing business understanding
- terminology
- ownership
- modelling limitations
- business drivers

Readers unfamiliar with the domain should understand:

Why did this modelling discussion become necessary?

Business Problem

The Business Problem should describe:

- business ambiguity
- ownership confusion
- inconsistent terminology
- unclear responsibilities

Poor.

```
Playback package too large.
```

Better.

```
Playback currently owns two unrelated business concepts.
```

The domain should describe the business.

Not the package structure.

Options

Every Domain ADR SHOULD document alternative models.

Example.

```
Collections
```

↓

```
Library Context
```

versus.

```
Collections
```

↓

```
Independent Context
```

Every alternative should include:

- benefits
- drawbacks
- business implications

Rejected models remain valuable knowledge.

Decision

The Decision section answers:

> ***How should the business now be modelled?***

The wording should remain concise.

Detailed implementation belongs elsewhere.

Consequences

Every modelling decision introduces trade-offs.

Example.

Choosing:

```
Playback
```

↓

```
Independent Context
```

Benefits.

- clearer ownership
- independent evolution
- simpler aggregates

Costs.

- additional events
- more explicit coordination
- increased context boundaries

Trade-offs should always be documented honestly.

Migration

Changing a Domain Model often requires migration.

Examples include:

- renamed events
- renamed terminology
- aggregate restructuring
- context boundaries
- repository changes

Migration guidance should explain:

- affected capabilities
- compatibility expectations
- rollout strategy

The Domain should evolve predictably.

Language Evolution

Changes to the ubiquitous language SHOULD always have an accompanying ADR.

Example.

```
Watch History
```

↓

```
Viewing History
```

The ADR should explain:

- why terminology changed
- expected business benefits
- affected specifications

Language is architecture.

Changing language changes understanding.

Domain Ownership

Changes affecting ownership SHOULD always be documented.

Examples.

```
Metadata

↓

Owns Artwork
```

instead of:

```
Library

↓

Owns Artwork
```

Ownership decisions have long-term architectural consequences.

Future contributors should understand the reasoning.

Context Maps

Changes to Context Maps SHOULD produce ADRs.

Example.

```
Recommendations

↓

Independent Context
```

This affects:

- ownership
- events
- extension boundaries
- runtime behaviour

Context relationships are architectural decisions.

Not implementation details.

Repository Structure

Recommended layout.

```
architecture/

    adrs/
```

```
ADR-001-library-context.md

ADR-002-playback-domain.md

ADR-003-metadata-ownership.md

ADR-004-continue-watching.md

ADR-005-recommendation-domain.md
```

Domain ADRs should remain close to the architectural specifications they support.

Review Process

Domain ADRs SHOULD receive architectural review.

Review should consider:

- business correctness
- language consistency
- ownership clarity
- future evolution
- compatibility
- architectural simplicity

The objective is improving the business model.

Not merely approving implementation.

Documentation

Accepted Domain ADRs SHOULD eventually be reflected within:

- MEG specifications
- architecture diagrams
- bounded context maps
- ubiquitous language
- contributor documentation

The Domain Model should remain internally consistent.

Documentation should evolve alongside it.

Mosaic Guidelines

Within Mosaic:

- Significant domain modelling decisions SHOULD have ADRs.
- ADRs MUST describe business reasoning.

- Ubiquitous Language changes SHOULD be documented.
- Context ownership changes SHOULD be documented.
- Alternatives MUST be considered.
- Trade-offs MUST be acknowledged.
- Historical ADRs MUST remain available.
- Domain evolution SHOULD remain deliberate.

Relationship to MEG

MEG-003 defines:

> ***How the business is modelled today.***

Domain ADRs explain:

> ***Why it is modelled that way.***

Together they preserve one of Mosaic's most valuable assets:

The understanding of the business itself.

Implementation can always change.

Business understanding is significantly harder to recover once lost.

Summary

A Domain Model represents years of accumulated understanding.

Without ADRs, that understanding gradually disappears.

Within Mosaic, Domain ADRs ensure that future contributors inherit not only the model itself, but also the reasoning that shaped it.

That reasoning is often considerably more valuable than the implementation built upon it.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`15-modelling-guidelines.md`

Next File

`17-contributor-guidance.md`

Contributor Guidance

> *Every contribution changes the model. The question is whether it improves our understanding of the business.*

Purpose

The Domain Model is the intellectual centre of the Mosaic platform.

Unlike infrastructure, which changes to support the business, the Domain Model exists to describe the business itself.

Every contributor therefore shares responsibility for protecting:

- business language
- business ownership
- business boundaries
- business behaviour

This document provides practical guidance for engineers contributing to the Mosaic Domain Model.

Philosophy

Within Mosaic:

> ***Protect the model before extending it.***

Adding functionality should never weaken:

- ubiquitous language
- aggregate boundaries
- bounded contexts
- business ownership
- domain consistency

The Domain Model should become clearer over time.

Never more complicated.

Before Writing Code

Before implementing a new feature ask:

- What business problem exists?
- Which Bounded Context owns it?
- Which Aggregate protects it?

- Which business rules apply?
- Which Domain Events naturally occur?

Implementation should begin only after these questions have clear answers.

Before Creating A New Domain Concept

Ask:

- Does this concept already exist?
- Is the language consistent?
- Does another Aggregate already own this behaviour?
- Is this genuinely a new business concept?

Avoid creating duplicate concepts with different names.

The ubiquitous language should remain consistent.

Before Creating A New Bounded Context

A new Bounded Context SHOULD only be introduced when:

- the language differs significantly
- ownership differs
- business rules differ
- independent evolution is desirable

A new package does **not** automatically justify a new Bounded Context.

Bounded Contexts represent business boundaries.

Not code organisation.

Before Creating An Entity

Ask:

> *Does the business recognise this concept through its identity?*

If yes:

Entity.

If no:

Consider a Value Object.

Identity should always have business meaning.

Before Creating A Value Object

Ask:

- Is identity irrelevant?
- Is the concept immutable?
- Does equality depend entirely upon value?

If the answer is yes:

Model a Value Object.

Do not introduce identity unnecessarily.

Before Creating An Aggregate

Ask:

> *Which business rules must always remain true together?*

Do **not** ask:

> *Which objects reference each other?*

Consistency determines Aggregate boundaries.

Object relationships do not.

Before Creating A Domain Service

A Domain Service should be the last modelling choice.

Ask:

- Can this behaviour belong to an Aggregate?
- Can it belong to a Value Object?
- Can it belong to an Entity?

Only if the answer is "no" should a Domain Service be introduced.

Large numbers of Domain Services usually indicate weak Aggregate modelling. [oai_citation:0‡Reddit](https://www.reddit.com/r/DomainDrivenDesign/comments/1sgqsv8/most_ddd_advice_starts_in_the_wrong_place/?utm_source=chatgpt.com)

Before Creating A Repository

Repositories should appear only after:

- the Aggregate exists
- the Aggregate Root is understood

- persistence requirements become necessary

Do not design repositories first.

Model first.

Persist later.

Before Creating A Domain Event

Ask:

- Did something important happen?
- Would the business describe this as a fact?
- Does another capability care about this fact?

If the event describes:

```
Do Something
```

it is probably a command.

If it describes:

```
Something Happened
```

it is probably a Domain Event.

Before Renaming A Business Concept

Changing language changes understanding.

Before renaming:

- update documentation
- update diagrams
- update events
- update repositories
- update package names where appropriate

Language should evolve consistently.

Half-completed terminology changes create architectural confusion.

Before Merging

Every Domain contribution SHOULD satisfy the following checklist.

Business Language

- Ubiquitous Language remains consistent.

- No duplicate terminology introduced.
- Names communicate business concepts.

Ownership

- Ownership remains explicit.
- Aggregate boundaries remain clear.
- Bounded Context responsibilities remain unchanged unless intentionally modified.

Behaviour

- Business behaviour remains inside the Domain.
- Aggregates enforce invariants.
- Domain Services remain focused.

Events

- Domain Events describe completed facts.
- Aggregate ownership remains correct.
- Event names reinforce business language.

Documentation

- MEG updated where required.
- ADR created for significant modelling changes.
- Context Maps updated where appropriate.
- Glossary updated when introducing new terminology.

The model and its documentation should evolve together.

Avoid Technical Thinking

During modelling discussions avoid asking:

- Which database table?
- Which HTTP endpoint?
- Which JSON schema?

Instead ask:

- What does the business call this?
- Who owns this concept?

- What behaviour exists?
- What rules always remain true?

The domain should remain independent of implementation.

Refactor The Model

Refactoring the Domain Model is encouraged.

Examples include:

- improving terminology
- refining Aggregate boundaries
- simplifying Value Objects
- extracting clearer concepts

Improving understanding is one of the primary goals of Domain-Driven Design.

Refactoring should therefore be viewed positively when it improves the model.

Review Mindset

Domain reviews should focus on:

- business correctness
- language
- ownership
- modelling clarity
- consistency
- maintainability

Questions such as:

> *"Does this model better represent the business?"*

are significantly more valuable than:

> *"Could this be implemented differently?"*

Implementation should support the model.

Not redefine it.

Domain Tests

Business behaviour should be verified through domain tests.

Examples include:

- Aggregate invariants
- Entity behaviour
- Value Object validation
- Domain Service decisions
- Domain Events

The Domain should be testable without:

- HTTP
- databases
- runtime infrastructure

Pure domain tests provide the fastest feedback and the clearest business validation.

Learning The Domain

New contributors SHOULD study the Domain Model in the following order.

```
Ubiquitous Language

↓

Subdomains

↓

Bounded Contexts

↓

Context Maps

↓

Aggregates

↓

Entities

↓

Value Objects

↓

Domain Services

↓

Domain Events
```

Understanding terminology first dramatically improves modelling quality.

Engineering Culture

Domain modelling should be collaborative.

Contributors are encouraged to:

- question terminology
- challenge unclear ownership
- simplify concepts
- refine language
- improve documentation
- discuss business behaviour with domain experts

The Domain Model should improve through conversation.

Not individual opinion.

Contributor Checklist

Before requesting review, confirm:

- [ ] The ubiquitous language remains consistent.
- [ ] Business ownership is explicit.
- [ ] Aggregate boundaries remain clear.
- [ ] Domain behaviour remains inside the domain.
- [ ] Invariants remain protected.
- [ ] Domain Events describe completed business facts.
- [ ] Infrastructure concerns have not leaked into the model.
- [ ] Documentation has been updated.
- [ ] The Domain Model is clearer than before.

Relationship to MEG

This document explains how contributors should evolve the Domain Model established throughout MEG-003.

The previous chapters define:

> ***How the business should be modelled.***

This chapter defines:

> ***How engineers should improve that model over time.***

Protecting the integrity of the Domain Model is a shared engineering responsibility.

Summary

The Domain Model is not simply another layer of the application.

It is the shared understanding upon which the entire platform is built.

Every contribution should strengthen:

- clarity
- ownership
- language
- correctness

Within Mosaic, the highest compliment a contributor can receive is not:

> *"That is clever."*

It is:

> ***"That models the business perfectly."***

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`16-adrs.md`

Next File

`glossary.md`

Glossary

> *A shared model requires a shared language. Every important business term should have one unambiguous meaning.*

Purpose

This glossary defines the terminology used throughout the Mosaic Domain-Driven Design specification.

The definitions contained within this document establish the canonical business vocabulary for:

- Architecture Specifications
- ADRs
- Source Code
- Documentation
- Extension SDKs
- Technical Discussions

Where a term has a specific meaning within the Mosaic Domain Model, that definition takes precedence over informal usage.

A

Aggregate

A consistency boundary consisting of:

- one Aggregate Root
- zero or more Entities
- zero or more Value Objects

An Aggregate protects business invariants.

It is persisted as a single unit.

Aggregate Root

The public entry point into an Aggregate.

The Aggregate Root:

- owns business consistency
- protects invariants
- exposes business behaviour
- is the only externally accessible object within the Aggregate

Every Aggregate has exactly one Aggregate Root.

B

Bounded Context

A boundary within which a Domain Model remains valid.

Every Bounded Context owns:

- language
- business rules
- terminology
- ownership

Different contexts may legitimately model similar concepts differently.

Business Rule

A rule describing valid business behaviour.

Business Rules belong inside the Domain Model.

They do not belong in:

- HTTP
- databases
- repositories
- infrastructure

C

Context Map

A description of the relationships between Bounded Contexts.

Context Maps identify:

- ownership
- dependency direction
- translation boundaries
- communication mechanisms

They describe architecture.

Not implementation.

Core Domain

The part of the business providing Mosaic's primary competitive advantage.

Core Domains receive the greatest architectural investment.

D

Domain

The area of knowledge the software exists to model.

Within Mosaic:

`Media Management`

is the overall domain.

Domain Event

An immutable record describing a completed business fact.

Domain Events originate inside the Domain Model.

They later become Runtime Events.

Domain Model

The collection of concepts representing the business.

The Domain Model includes:

- Entities
- Value Objects
- Aggregates
- Domain Services
- Domain Events

It intentionally excludes infrastructure.

Domain Service

Business behaviour that belongs to the Domain but belongs naturally to no individual Aggregate.

Domain Services are:

- stateless
- business focused
- behaviour oriented

E

Entity

A business concept defined by its identity.

Entities:

- possess stable identity
- evolve over time
- own behaviour

Changing state does not change identity.

F

Factory

A component responsible for constructing valid domain objects.

Factories ensure:

- invariants
- default state
- complete construction

Factories perform creation.

Not business behaviour.

G

Generic Domain

A supporting capability that does not differentiate the Mosaic platform.

Examples include:

- logging
- configuration
- scheduling

Generic Domains should usually leverage established solutions.

I

Identity

The stable characteristic that distinguishes one Entity from another.

Identity belongs to the business.

Not the database.

Invariant

A business rule that must always remain true.

Aggregates exist primarily to protect invariants.

P

Published Language

A stable contract through which Bounded Contexts communicate.

Within Mosaic this is usually expressed through:

- Domain Events
- Runtime Events
- public interfaces

R

Repository

A persistence abstraction responsible for loading and saving Aggregate Roots.

Repositories hide storage implementation.

They expose business concepts.

S

Shared Kernel

A shared Domain Model used by multiple Bounded Contexts.

Within Mosaic this pattern is generally discouraged because it increases coupling.

Published contracts are preferred.

Subdomain

A distinct business capability within the larger Domain.

Examples include:

- Playback
- Library
- Metadata
- Search

Each Subdomain owns one coherent area of business responsibility.

Supporting Domain

A business capability that enables the Core Domain but does not define Mosaic's competitive advantage.

Examples include:

- Authentication
- Notifications
- Search

U

Ubiquitous Language

The shared vocabulary used by:

- engineers
- architects
- product owners
- documentation
- source code

Every business concept should have one canonical meaning within its Bounded Context.

V

Value Object

A business concept defined entirely by its value.

Value Objects:

- possess no identity
- should be immutable

- may contain behaviour
- reinforce the ubiquitous language

Examples include:

- Duration
- Resolution
- Language

Common Acronyms

Acronym	Meaning
ACL	Anti-Corruption Layer
ADR	Architectural Decision Record
DDD	Domain-Driven Design
MEG	Mosaic Engineering Guidelines
MDS	Mosaic Design Specifications
MDL	Mosaic Design Language
UBL	Ubiquitous Language

Relationship to MEG-003

This glossary supports every document within the Domain-Driven Design specification.

Definitions should remain consistent across:

- Domain Models
- Architecture Specifications
- ADRs
- Runtime Specifications
- Extension SDKs
- Contributor Documentation

Whenever the ubiquitous language evolves, this glossary SHOULD be updated before introducing new terminology elsewhere.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`17-contributor-guidance.md`

Next File

`references.md`

References

> *The Mosaic Domain Model is built upon established principles of Domain-Driven Design, adapted for an event-driven, extension-first platform.*

Purpose

This document records the primary references that informed the Domain-Driven Design principles established throughout MEG-003.

The objective of these references is not to prescribe implementation.

Instead, they provide the theoretical and architectural foundations upon which the Mosaic Domain Model has been developed.

Where the MEG differs from traditional DDD guidance, it does so intentionally to support the unique requirements of the Mosaic platform.

Primary References

Domain-Driven Design

Author

Eric Evans

Purpose

The foundational work introducing:

- Ubiquitous Language
- Bounded Contexts
- Aggregates
- Entities
- Value Objects
- Domain Services
- Strategic Design
- Tactical Design

This work forms the conceptual foundation of MEG-003.

Although Mosaic adapts certain implementation details for an event-driven architecture, the underlying modelling philosophy remains strongly aligned.

Domain-Driven Design Reference

Author

Eric Evans

Purpose

A concise summary of the fundamental principles introduced in Domain-Driven Design.

Recommended reading for all contributors.

URL

https://domainlanguage.com/ddd/reference/

Implementing Domain-Driven Design

Author

Vaughn Vernon

Purpose

Expands practical implementation guidance including:

- Aggregate design
- Aggregate boundaries
- Repositories
- Factories
- Domain Events
- Bounded Contexts

Many tactical recommendations adopted within Mosaic align closely with this work.

Effective Aggregate Design

Author

Vaughn Vernon

Purpose

A three-part series focusing specifically on:

- Aggregate sizing
- Consistency boundaries
- Aggregate relationships

This work significantly influenced the Aggregate guidance within MEG-003.

URL

https://dddcommunity.org/library/vernon_2011/

Supporting References

Martin Fowler

Martin Fowler's writings on Domain-Driven Design significantly influenced the modelling approach used throughout Mosaic.

Recommended topics include:

- Aggregates
- Value Objects
- Bounded Contexts
- Domain Events
- Anti-Corruption Layers

URL

https://martinfowler.com/

Patterns of Enterprise Application Architecture

Author

Martin Fowler

Relevant concepts include:

- Repository
- Unit of Work
- Identity Map
- Data Mapper

Mosaic intentionally adopts some of these concepts while deliberately avoiding others where they introduce unnecessary complexity.

Software Engineering References

Clean Architecture

Author

Robert C. Martin

Referenced primarily for:

- dependency direction
- separation of concerns
- business independence

Implementation details within Mosaic differ where Domain-Driven Design provides more appropriate guidance.

Refactoring

Author

Martin Fowler

Relevant for:

- evolving models
- improving terminology
- incremental architectural refinement

The Domain Model is expected to evolve continuously.

This work strongly supports that philosophy.

The Pragmatic Programmer

Authors

Andrew Hunt

David Thomas

Referenced primarily for:

- continuous improvement
- engineering discipline
- incremental design

Many contributor guidelines throughout the MEG reflect the engineering mindset encouraged by this work.

Event-Driven Architecture

Although MEG-003 focuses upon the Domain rather than the Runtime, Domain Events naturally integrate with the runtime architecture defined in MEG-002.

Recommended references include:

Martin Fowler

Event-Driven Architecture

https://martinfowler.com/articles/201701-event-driven.html

Enterprise Integration Patterns

Gregor Hohpe

Bobby Woolf

Useful concepts include:

- Message Channels
- Published Language
- Message Translation

These patterns complement Domain Events without influencing the Domain Model itself.

Go References

The Domain Model intentionally remains implementation independent.

However, contributors are encouraged to understand how Go supports rich domain modelling.

Recommended references include:

Effective Go

https://go.dev/doc/effective_go

Go Code Review Comments

https://go.dev/wiki/CodeReviewComments

Go Blog

Recommended topics:

- composition
- interfaces
- package design
- concurrency

Go implementation should always support the Domain Model.

Never define it.

Internal Mosaic Specifications

The following specifications complement MEG-003.

Engineering

- MEG-001 Go Engineering Standards
- MEG-002 Event-Driven Runtime

Planned Engineering Specifications

- MEG-004 Hexagonal Architecture

- MEG-005 Extension Platform
- MEG-006 Runtime Architecture
- MEG-007 Storage Architecture
- MEG-008 Observability
- MEG-009 Security
- MEG-010 Performance Engineering

Mosaic Design Language

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

Mosaic Design Specifications

- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography System
- MDS-005 Motion System
- MDS-006 Composition Engine
- MDS-007 Tile Framework
- MDS-008 Component Library

Domain Principles

The Domain Model established throughout MEG-003 is intentionally built upon several enduring principles.

These include:

- Business before technology.
- Language before implementation.
- Behaviour before data.
- Identity before persistence.
- Consistency before convenience.
- Small Aggregates.

- Explicit ownership.
- Evolutionary modelling.
- Infrastructure independence.
- Rich domain behaviour.

These principles should remain stable even as implementation evolves.

Keeping References Current

Domain modelling continues to evolve.

Software architecture continues to evolve.

The references contained within this document SHOULD therefore be reviewed periodically to ensure that:

- recommended material remains current
- obsolete guidance is removed
- new influential work is incorporated
- architectural thinking remains contemporary

The philosophy of the Domain Model should remain stable even as modelling techniques improve.

Closing Statement

MEG-003 does not attempt to replace Domain-Driven Design.

Instead, it adapts proven modelling principles into an architecture specifically designed for the Mosaic platform.

The resulting Domain Model intentionally emphasises:

- business language
- explicit ownership
- evolutionary design
- autonomous capabilities
- clear consistency boundaries

Everything else within the platform exists to support these business concepts.

Because ultimately:

> ***The business is the product.***

The software merely gives it form.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`glossary.md`

Next File

End of Specification